

项目文档v0.1.1

摘要

《DeltaNights》是一款以PvPvE“搜打撤”模式为主的TPS多人战术竞技游戏，以二次元科幻漫画风格作为主要表现形式，面向爱好二次元风格与团队战术策略的战术竞技玩家。与主流“搜打撤”相比，它更侧重于画面的表现形式、角色技能的组合与差异化、攻击形式的多元化与对战撤离策略，为玩家提供前所未有的沉浸体验。

关键词：搜打撤，PvPvE，二次元

日期	修改	描述
2025.11.4	0.0.1	构建文档框架
2025.11.23	0.1.0	总结需求文档等文档内容到此文档
2025.12.10	0.1.1	完善需求分析

git提交名	github账号	学号
SL_Skuy	SLSkuy	2023051604044
星焰	StarFlameDisorder	2023051604032
JMSH	YOUSAFOREVER	2023051604049
kish666	kishizi	2023051604046
Warfarinrl	warfarinrl	2023051604050
Rain	MuziRain-official	2023051604020

1. 立项

1.1 项目起源与提案

1.1.1 项目背景

近年来FPS类游戏市场竞争激烈，以“搜打撤”（Search - Fight - Evacuate）模式为主要玩法的战术类游戏成为市场主流。但该赛道仍存在明显空白--缺乏将 二次元风格 与 战术竞技机制 深度融合的产品。本项目旨在补缺市场漏洞，构建一个融合团队战术、角色技能与策略机制的竞技体验。

1.1.2 商机与竞争机会

- 市场缺口：搜打撤模式与二次元的交叉生态位存在空白
- 玩家群体：喜好二次元与竞技游戏的玩家数量高，潜在用户基数大
- 内容创新：传统TPS游戏缺乏角色差异化、攻击形式单一
- 近似游戏：《卡拉彼丘》运营状况较差产能低下，《APEX》反作弊系统差，游玩体验糟糕，《命运扳机》潜在玩家较多，《三角洲行动》的运营决策失误导致玩家流失，《守望先锋》因角色与技能数过多且高度依赖团队配合导致上手门槛高。《无畏契约》因游戏模式过于单一且竞技性强导致玩家无法长期留存。

1.1.3 项目定位

《DeltaNights》是一款以PvPvE“搜打撤”模式为主的TPS多人战术竞技游戏，以二次元科幻漫画风格作为主要表现形式，面向爱好二次元风格与团队战术策略的战术竞技玩家。与主流“搜打撤”相比，它更侧重于画面的表现形式、角色技能的组合与差异化、攻击形式的多元化与对战撤离策略，为玩家提供前所未有的沉浸体验。

1.1.4 核心循环

核心玩法为 PvPvE “搜 - 打 - 撤”：

- 搜（Search）：玩家在地图中搜索战利品与资源；
- 打（Fight）：与敌方玩家及AI敌人进行战斗；
- 撤（Evacuate）：在安全区撤离并带走搜集成果。

1.1.5 系统组成

- 客户端：角色控制、UI交互、语音通信、技能释放、动画展示；
- 服务端：匹配、战斗同步、数据统计、经济系统；
- 第三方服务：反作弊服务、云平台、语音与存储服务。

1.2 Business Case

1.2.1 市场机会与规模

- 目标用户群体明确：二次元+战术竞技双栖玩家，具备高付费意愿与社区活跃度。
- 市场缺口显著：目前尚无成功融合“二次元美术 + 搜打撒玩法 + 角色技能系统”的产品。
- 竞争环境分析：
 - 《APEX》反作弊差，《卡拉彼丘》运营乏力，《三角洲行动》决策失误——均未完全满足目标用户需求。
 - 《DeltaNights》通过美术风格、角色技能、攻击形式、战术标记系统等维度实现差异化。

1.2.2 收益预测

- 主要收入来源：
 - 角色皮肤、武器外观、表情动作等虚拟商品销售
 - 战斗通行证
 - 高级礼包

1.2.3 风险评估与缓解策略

风险	影响	缓解策略
市场竞争加剧	中	快速迭代、强化社区运营、持续内容更新
技术实现难度	高	采用Unity引擎、分阶段开发、原型验证
玩家接受度低	中	早期测试反馈、差异化宣传

1.2.4 关键成功因素

- 清晰的愿景与定位：明确“二次元战术竞技”细分市场
- 持续的内容更新：保持玩家活跃与社区热度
- 强反作弊与公平性：保障竞技环境健康
- 社区驱动开发：通过玩家反馈优化体验

2. 愿景

2.1 问题陈述

- 产品类型的同质化：当前市场的竞争十分激烈，开发厂商急于从市场竞争中蹭热度赚一笔，多数倾向于对成功案例的粗劣换皮模仿，产品缺乏创造性、创新性、有趣性，使得市面上的产品同质化严重

- 产品的互动性较低：互动形式局限于固定表情/快捷语音，缺乏深度的合作机制、物品获得受算法机制影响，使得用户体验局限于算法框架之下，自由度较低，可互动性较弱，大幅降低用户从随机事件中获取到的刺激感
- 策略性：缺乏有效的队内沟通工具和战术标记系统，团队配合成本高、游戏内教学引导不足，核心机制未被清晰传达，导致学习曲线陡峭
- PVP和PVE：PVP模式受玩家对于产品的熟练度影响，游戏对局的匹配机制可能出现高熟练玩家与低熟练玩家竞争，迫使双方从中获取的正反馈降低；PVE模式受敌人的AI算法影响，过高的或过低的敌人AI算法都会大幅降低用户从中获取到的正反馈
- 玩家流失：若产品内容在长期内都得不到更新迭代，玩家可能会因重复体验相同的内容而感到乏味；适度提高竞技强度可以提高玩家的兴奋感，但过高的竞技强度会导致休闲玩家从中获取到的正反馈降低，进而导致玩家流失

要素	描述
问题	市场竞争激烈，创新不足、缺乏深度的合作机制、物品获得过于随机、团队配合成本高、游戏内教学引导不足、匹配机制、AI行为模式固定、刷新点背板
影响	玩家、开发团队、内容创作者、运营团队
结果	玩家流失、新玩家减少
解决方案	加强创新和调研，开放测试服，将做出的更改提前试验在一部分玩家身上，通过反馈做出决定

2.2 涉众与用户

2.2.1 涉众

2.2.1.1 涉众类型与代表

类型	代表	简述	关注点
项目发起者	投资方、制作人	项目资金与方向决策	确定项目能够按期开发上线
开发团队	策划、程序、美术、测试	实际开发游戏	项目成功、技术实现、创意落地
用户	玩家、主播、电竞选手	直接参与游戏的游玩的玩家	社交与策略
运维	维护团队、后续开发团队	保证游戏正常运转、服务器维护、漏洞修复和更新	稳定性、更新速度质量
服务商	服务器提供商、游戏平台、媒体、反作弊	提供游戏的运行基础、宣发、提供	市场份额、资金回报

运营	宣发、客服	添加游戏活动、玩家账号维护、媒体官号维护	用户留存、服务器稳定、活动效果
支持性涉众	自媒体创作者、社区、媒体	维护游戏生态、宣传游戏	用户增加和留存

2.2.1.2 涉众优先级

涉众	权力	兴趣	优先级	沟通策略
投资方	高	高	高	定期汇报与决策会议
玩家	中	高	高	社区反馈与测试活动
开发团队	高	高	高	敏捷迭代同步
合作者	中	中	中	定期沟通与协议签订
运维	高	中	中	技术同步与维护报告
支持性涉众	低	中	低	提供创作素材与活动参与

2.2.2 用户

用户类型	特征	核心需求
玩家	关注社交游戏体验、追求更高的操作技术	易上手、团队协作、情绪价值、操作性与深度策略
运营人员	关注游戏运营情况，对运营策略做出调整	让公司尽可能获利的前提下满足用户需求
运维工程师	修复游戏出现bug，定期维护服务器	bug及时反馈及时修复，避免用户出现不愉快体验
游戏策划	思考游戏未来方向，为玩家游戏体验做出优化	能够提供合理的游戏未来开发方案给开发团队，使游戏体验得到优化，使游戏内容更加丰富
系统管理员	获取系统日志，局内反作弊监管	保证游戏日志稳定输出，对可疑战局采取人为监管，利用系统服务运营与玩家

2.3 关键涉众和用户的需要

- 核心玩家 - “二次元战术家”

- 特征： 年龄18-28岁，同时是二次元文化爱好者与硬核竞技游戏玩家。追求角色个性、视觉风格与深度策略的结合。
- 核心诉求： 强烈的角色认同感、丰富的战术组合空间、高回报的团队协作体验、公平且有深度的竞技环境。
- 游戏动机： 钻研BD（角色构建）、与固定队达成战术默契、在竞技中获得优越感、收集个性化外观。
- 潜力玩家 - “风格化探索者”
 - 特征： 来自传统竞技游戏或二次元RPG领域，被独特的二次元科幻美术风格所吸引，但对“搜打撤”模式可能较为陌生。
 - 核心诉求： 友好的上手体验、直观的视觉反馈、丰富的角色与攻击形式、可持续的成长目标。
 - 游戏动机： 体验新颖玩法、享受视觉盛宴、完成角色与装备收集、与朋友进行轻度团队合作。

2.4 产品概述

2.4.1 产品定位陈述

项目	内容
for(目标)	二次元文化与战术竞技游戏爱好者，年龄18-28岁，追求角色个性与团队策略
who(需求)	对现有游戏的美术风格、角色差异化与策略深度不满，渴望更具个性与协作性的游戏体验
the(产品名称)	《DeltaNights》是一款二次元风格PVPVE“搜打撤”战术竞技游戏
that(核心价值)	提供融合角色技能、多元攻击与动态撤离策略的团队PVPVE体验
unlike(竞品)	《APEX》 《卡拉彼丘》 《三角洲行动》
Our product (差异化)	更强调角色技能组合、画面表现力、战术自由度与持续内容更新

2.4.2 完整的产品概述

能力概述

《DeltaNights》是一款二次元风格PVPVE“搜打撤”战术竞技游戏。核心能力包括：

- 角色战术系统： 操作技能独特的干员，实现团队互补与环境互动。
- 多元战斗探索： 融合枪械、技能等多元素，进行资源搜索与动态PvE挑战。
- 动态撤离机制： 通过守护信标或争夺公共点完成策略性撤离。
- 成长与收集： 提供局外技能解锁与丰富的外观收藏。

客户效益

- 沉浸体验：独特二次元美术风格带来强烈视觉享受与角色认同。
- 策略成就感：丰富的战术组合与高价值目标带来高强度正反馈。
- 友好体验：智能标记系统降低沟通门槛，动态事件确保对局新鲜感。

竞争定位

- 本产品核心竞争力在于“二次元画风 + 角色技能驱动 + PVPVE搜打撤”的完整融合，以玩家体验为核心，创造了独特的细分市场定位。

2.5 产品特性

编号	特性名称	描述	优先级
F01	核心射击系统	第一/第三人称自由切换，包含枪械、手雷等武器系统	必须
F02	搜打撤机制	在地图中搜索战利品、击败敌人、激活撤离点	必须
F03	角色技能系统	各角色拥有被动、普通与终极技能	必须
F04	多人匹配与排位	实时联网匹配战斗	必须
F05	经济与仓库系统	玩家撤离携带的战利品进入个人仓库，可用于交易或制作	必须
F06	反作弊与公平竞技	集成反作弊系统，对开挂账号的严厉惩罚，保障竞技公平性	必须
F07	战术标记系统	支持快速标记敌人位置、战利品、战术指令降低语音沟通门槛	应该
F08	新手引导与训练场	沉引导玩家掌握“搜打撤”核心循环，提供可测试所有枪械和角色技能的训练场	应该
F09	对局回放系统	查询对局细节，死亡回防等功能，以供玩家复盘战术漏洞	应该
F10	角色/武器皮肤	提供不影响平衡性的个性化外观，是重要的商业化手段	应该

2.6 其他产品需求

- 性能需求
 - a. 响应时间：95% 的页面加载时间应在 2 秒以内
 - b. 吞吐量：系统应能支持每秒 1000 个并发用户登录
 - c. 容量：数据库应能存储至少 5 年的用户使用记录
- 安全性需求

- a. 身份认证：支持双因素认证（2FA）
 - b. 数据加密：所有用户密码必须以加盐哈希方式存储；传输数据必须使用 TLS 1.2+ 加密
 - c. 权限控制：实现基于角色的访问控制（RBAC），区分管理员、运营、普通用户权限
 - d. 合规性：个人数据处理必须符合 GDPR/《个人信息保护法》要求，提供用户数据导出和删除功能
 - 兼容性需求
 - a. 操作系统/设备：支持 pc、手柄和移动端；需适配平板电脑的横屏与竖屏模式
 - 可靠性、可用性和可拓展性需求
 - a. 可靠性：系统月度正常运行时间不低于 99.9%
 - b. 可用性：任何计划内的维护需提前 24 小时通知用户，且每月计划内停机时间不得超过 4 小时
 - c. 可扩展性：架构设计应支持在 2 小时内通过增加服务器节点，将系统处理能力提升 50%
 - 运营与监控需求
 - a. 日志：系统需记录所有关键用户操作和安全事件，并集中存储至少 90 天
 - b. 监控：需配置关键业务指标（如API 响应时间）的监控仪表盘和报警机制
 - c. 后台：运营人员需可通过管理后台对用户内容进行审核和封禁操作
 - 国际化与本地化需求
 - a. 国际化：代码架构需支持多语言，字符串需外部化处理
 - b. 本地化：首发支持简体中文和英文；UI 布局需适应不同语言的长度（如德语通常更长）
 - 法律与合规需求
 - a. 隐私政策必须明确告知用户数据收集和使用方式，并获取用户同意
 - b. 在欧盟地区提供服务，必须遵守 GDPR 规定
-

3. 用况建模

3.1 参与者确定

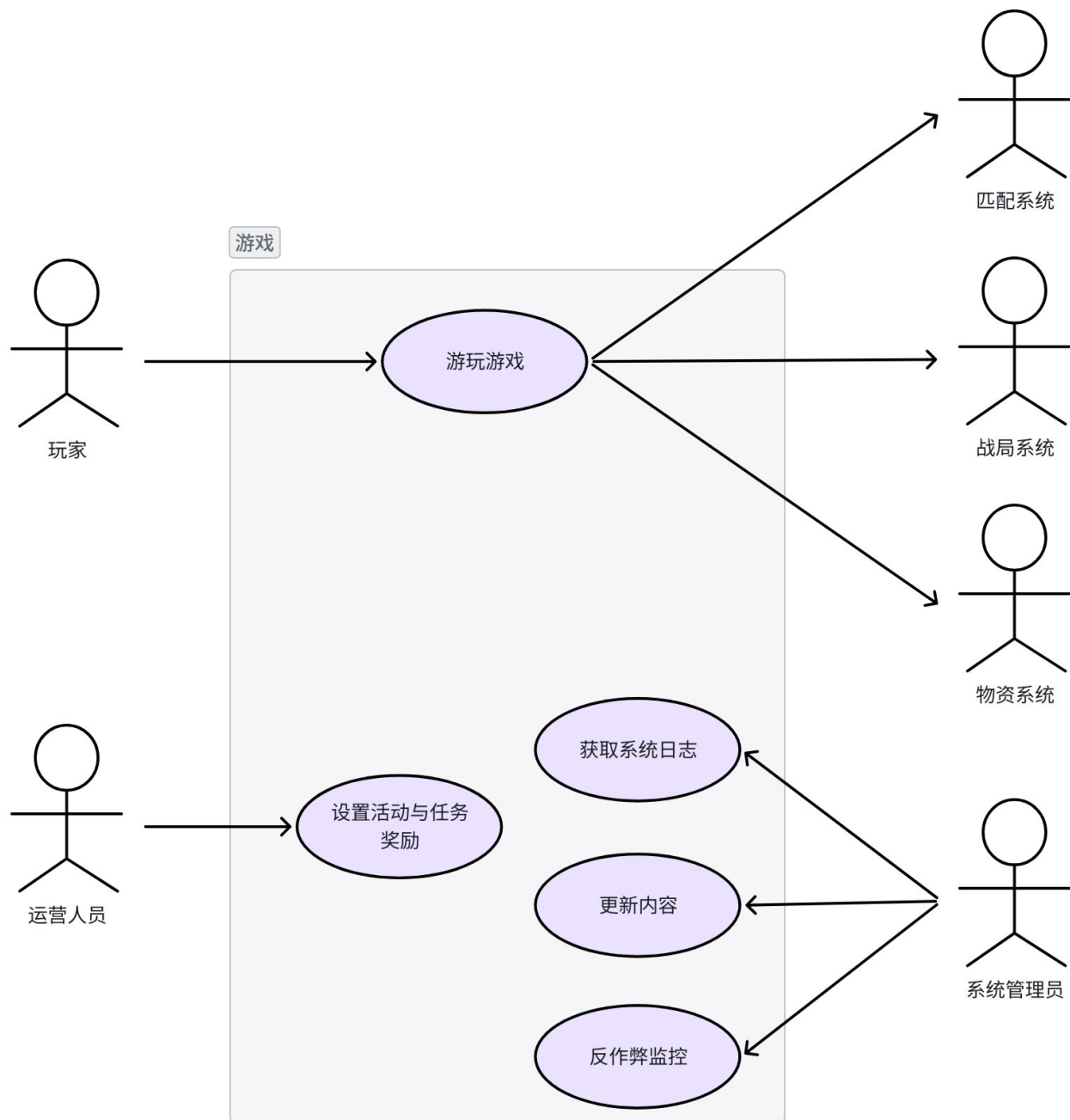
- 玩家（Player）：游戏的直接用户，执行战术、拾取资源、释放技能、撤离战区
- 匹配系统（Matchmaker）：执行战局匹配与房间分配逻辑
- 战局服务器（Battle Server）：处理实时战斗逻辑、AI逻辑、状态同步、战局结算等功能
- 运营人员（Operator）：设计游戏活动，配置任务奖励，关注玩家游玩体验
- 系统管理员（Administrator）：监控与维护服务器运行状态，保护数据安全

- 战局系统（War Situation system）：分配玩家起始位置，分配地图资源分布
- 物资系统（Material system）：管理物资信息
- 社交系统（Socialize System）：处理玩家与玩家之间的交互

3.2 术语表

名称	定义
玩家	游戏的直接用户，能够执行登录、匹配、战斗、交易等所有游戏操作。在战局中，玩家操控一个特定的游戏角色进行活动
战局	也称“对战局”。指一名或一组玩家从匹配成功、进入特定游戏地图开始，到最终成功撤离、全体阵亡或对局时间结束为止的完整、独立的一局游戏。
匹配系统	负责组织和管理战局中队伍的后台服务系统，将匹配成功的玩家组分配到指定的战局服务器中
战局服务器	运行战局的服务器，负责托管并执行特定战局内的所有实时游戏逻辑
战局系统	对战局内的资源，任务和玩家进行分配
物资系统	负责管理玩家在局内，局外获取到的物资
撤离	玩家在战局内抵达撤离点，在倒计时结束后便完成撤离

3.3 用况图



3.4 用况详细描述

游玩游戏

参与者：玩家

前置条件：

- 玩家网络和服务器网络可用

基本流：

1. 执行子流S1 登录
2. 执行子流S2 组队
3. 执行子流S3 匹配

4. 执行子流S4 生成战局
5. 执行子流S5 战局内交互
6. 执行子流S6 结算对局
7. 重复步骤2-6，直到玩家退出
8. 结束

子流：

子流S1 登录

参与者： 用户，登录系统

前置条件：

- 用户已启动游戏应用，并进入登录界面。

事件流：

1. 用户在登录界面选择登录方式
2. 系统根据用户选择提供登录验证
3. 用户提供登录验证凭证
4. 系统验证凭证通过后，加载用户个人资料和游戏数据，并跳转至游戏首页。
5. 系统显示“登录成功”提示信息
6. 系统修改用户状态
7. 用例终止。

备选流：

A1 “处理无效凭据”（特定备选流）

扩展点及触发条件： 在基本流第3步，当系统验证发现用户名或密码错误时。

事件流：

1. 系统提示“用户名或密码错误，请重试”。
2. 系统清空密码输入框，保留用户名输入框内容。
3. 事件流恢复：事件流恢复到基本流第1步，用户可重新输入凭据或终止登录。

A2 “处理网络或服务器错误”（特定备选流）

扩展点及触发条件： 在基本流第3步，当系统因网络连接失败或服务器无响应导致登录请求失败时。

事件流：

1. 系统提示“网络连接失败，请检查网络后重试”。
 2. 事件流恢复：事件流恢复到基本流第1步，用户可重新尝试登录或终止。
-

子流S2 组队

参与者：用户，匹配系统，社交系统

前置条件：

- 用户登录成功

事件流：

1. 用户在首页或活动界面点击“组队”功能之后，用况开始启动。
2. 系统加载组队界面，展示可加入队伍列表、创建队伍入口等信息。
3. 用户选择执行以下任一操作：
 - 创建新队伍
 - 加入现有队伍
 - 向现有队伍发送加入申请

{执行组队操作}

4. 系统根据用户所选操作进行处理：
 - 创建队伍：系统要求填写队伍名称、简介、人数上限等并创建队伍。
 - 加入队伍：系统将用户加入该队伍（若该队伍无需审核）。
 - 申请加入：系统向队长/管理员发送入队申请。
5. 系统更新用户的队伍状态，并显示加入成功、创建成功或申请已发送等反馈信息。
6. 用况终止。

备选流：

A1 “处理队伍已满”（特定备选流）

扩展点及触发条件：在 {执行组队操作} 扩展点上，当用户尝试加入的队伍人数已满。

事件流：

1. 系统提示“队伍人数已满，无法加入”。
2. 系统返回队伍列表页面。

事件流终止：组队用况终止。

A2 “处理组队操作失败”（特定备选流）

扩展点及触发条件：

在 {执行组队操作} 扩展点上，当系统执行加入、申请、创建队伍时由于网络错误、权限问题等导致失败。

事件流：

1. 系统提示“操作失败，请重试”。

事件流恢复：事件流恢复到基本流第3步，用户可重新选择组队方式或终止。

子流S3 匹配

参与者： 用户，匹配系统

前置条件：

- 用户登录成功，并处于游戏大厅或匹配入口界面。
- 用户已满足匹配条件（例如，已完成组队或选择单人匹配）。

事件流：

1. 用户在大厅界面点击“开始匹配”按钮后，用况开始启动。
2. 系统加载匹配界面，显示匹配队列状态（如预计等待时间、当前队列人数）。
3. 系统根据用户匹配请求（如模式选择、队伍状态）加入匹配队列。
4. 系统在队列中寻找符合条件的其他玩家，并进行匹配计算。

{执行匹配操作}

5. 系统成功匹配到足够玩家后，创建游戏房间，并分配队伍。
6. 系统跳转至游戏加载界面，显示匹配成功信息（如队友列表、地图信息）。
7. 用况终止。

备选流：

A1 “处理匹配取消”（特定备选流）

扩展点及触发条件： 在 {执行匹配操作} 扩展点上，当用户主动取消匹配（如点击“取消匹配”按钮）时。

事件流：

1. 系统立即将用户移出匹配队列。
2. 系统提示“匹配已取消”。
3. 事件流终止：匹配用况终止，返回大厅界面。

A2 “处理匹配超时”（特定备选流）

扩展点及触发条件： 在 {执行匹配操作} 扩展点上，当匹配等待时间超过预设阈值（如5分钟）仍未成功时。

事件流：

1. 系统提示“匹配超时，请重新尝试”。
2. 系统自动将用户移出匹配队列。
3. 事件流恢复：事件流恢复到基本流第1步，用户可重新发起匹配或终止。

A3 “处理匹配失败”（特定备选流）

扩展点及触发条件： 在 {执行匹配操作} 扩展点上，当系统因错误（如服务器故障、数据不一致）导致匹配失败时。

事件流：

1. 系统提示“匹配失败，请重试”。
 2. 系统自动将用户移出匹配队列。
 3. 事件流恢复：事件流恢复到基本流第1步，用户可重新发起匹配或终止。
-

子流S4 生成战局

参与者：战局系统

前置条件：

- 战局匹配成功
- 服务器可用

事件流：

1. 系统分配游戏服务器资源
2. 系统加载游戏规则
3. 执行 S1: 初始化游戏地图
4. 玩家选择干员
5. 分配玩家位置
6. 系统初始化玩家干员
7. 系统确认所有玩家准备状态
8. 游戏对局正式开始
9. 结束

备选流：

A1 "生成地图错误"（特定备选流）

扩展点及触发条件： 在{初始化游戏地图}扩展点上，当系统初始化游戏地图发生错误时

事件流：

1. 系统操作：延长玩家在加载界面的时间
2. 系统重新初始化地图
3. 事件流恢复：玩家进入战局或玩家回到匹配界面

A2 "玩家中途退出"（特定备选流）

扩展点及触发条件： 在{玩家选择干员-结束}扩展点上，当玩家意外退出对局时

事件流：

1. 系统保留玩家当前信息
 2. 系统删除对应玩家操作角色及位置
 3. 在玩家重连时恢复保留信息，继续流程
-

子流S5 战局内交互

参与者：玩家、战局系统，系统管理员

前置条件：

- 对局已生成完备
- 玩家正常联机

事件流：

1. 系统接收玩家操作输入（移动、攻击、使用技能等）
2. 系统验证操作合法性
3. 系统处理操作结果(执行S1或S2或S3)
4. 系统更新和同步游戏状态
5. 重复步骤1-4，直到游戏结束条件满足
6. 结束

子流：

- S1 处理战斗操作:

计算状态效果

计算伤害

更新状态和血量

- S2 处理道具使用:

验证道具

应用效果

- S3 处理团队协作:

验证团队沟通信息

转发团队沟通信息

备选流

A1 "操作验证失败"

扩展点及触发条件：在{系统验证玩家操作合法性}扩展点上，当玩家操作输入不合法时
事件流：

- 1. 系统处理：拒绝操作，提示拒绝原因
- 2. 事件流恢复：返回等待新操作

A2 "反作弊检测触发"

扩展点及触发条件：在{系统验证玩家操作合法性}扩展点上，当玩家操作与实际输入不符合
事件流：

- 1. 系统处理：返回当前玩家信息
- 2. 系统将信息提交至系统管理员
- 3. 系统管理员对该玩家进行检测
- 4. 系统记录日志
- 5. 系统对该玩家做出相应操作

A3 "玩家阵亡"

扩展点及触发条件：在{系统处理操作结果(执行S1或S2或S3)}扩展点上，当玩家血量归零时
事件流：

- 1. 停止对玩家干员的操作
- 2. 返回玩家被淘汰面板
- 3. 等待玩家操作（观战，离开等）
- 4. 事件流恢复：继续战局流程

子流S6 结算对局

参与者：战局系统、物资系统

前置条件：

- 对局已结束
- 玩家撤离
- 玩家阵亡

事件流：

- 1. 系统收集战局最终数据
- 2. 执行 S1: 计算基础奖励

- 3. 执行 S2: 计算加成
- 4. 更新玩家统计
- 5. 系统调用物资系统发放奖励
- 6. 执行 S3:验证发放
- 7. 系统生成战局报告
- 8. 结束

子流:

- S1 计算基础奖励:

根据胜负结果计算基础奖励

- S2 计算加成:

根据玩家表现和活动计算额外奖励

- S3 验证奖励发放

向物资系统确认物资发放状况

备选流:

A1 "奖励发放失败"

扩展点及触发条件: 当{执行 S3:验证发放}扩展点上, 验证到发放错误或未发放到玩家仓库或邮件中时

事件流:

- 1. 物资系统重新对其发放
- 2. 再次验证是否成功
- 3. 记录错误日志
- 4. 事件流恢复: 继续正常流程

获取系统日志

参与者: 系统管理员

前置条件:

- 明确获取需求
- 具备访问权限

基本流:

- 1. 接收并确认请求: 系统管理员接收到获取日志的请求, 并明确需求细节。

{查看有无权限访问}

2. 定位日志文件：根据系统类型和应用程序配置，找到目标日志文件的存储路径。（满足条件进入子流S1）

{查看文件是否存在}

3. 确定时间范围：根据时间戳过滤出指定时间范围内的日志条目。
4. 检索与打包日志：将过滤后的日志输出重定向到一个临时文件。
5. 传输至本地：将打包好的日志文件从目标系统传输到本地分析环境或指定的安全存储区。
6. 执行初步检查：对下载的日志文件进行快速检查，例如验证文件完整性（未损坏）、确认时间范围是否正确、是否包含预期的内容（如相关的错误信息）。
7. 交付日志并记录：将日志文件交付给请求方（如开发工程师）。在工单或工作日志中记录此次操作。
8. 结束。

备选流：

A1 “权限不足”（特定备选流）

扩展点及触发条件：在{查看有无权限访问}扩展点上，发现系统管理员没有访问日志权限。

事件流：

1. 尝试使用 `sudo` 提权（如果该管理员拥有sudo权限）。
2. 如果权限依然不足，联系安全团队或系统所有者申请临时提升的权限。
3. 如果情况紧急且符合安全策略，可能有预先设定的“应急响应”流程可以触发，以获得临时访问权。

事件流恢复：事件流恢复到基本流第1步，重新检查是否有权限

A2 “文件不存在”（特定备选流）

扩展点及触发条件：在{查看文件是否存在}扩展点上，发现日志文件不存在。

事件流：

1. 检查日志文件路径是否正确，文件是否存在
2. 联系该系统的负责开发或运维人员，确认日志的正确位置。

事件流恢复：事件流恢复到基本流第2步，重新检查文件是否存在

子流S1：从集中式日志平台获取

参与者：系统管理员

前置条件：

- 使用了ELK等集中式日志系统

基本流：

1. 登录到日志平台的Web管理界面。
 2. 使用查询语言（KQL，SPL，LogQL）构建查询语句，指定主机、应用和时间范围。
 3. 执行查询并验证结果。
 4. 将查询结果导出为JSON、CSV或文本文件。
 5. 用况中止。
-

更新内容

参与者：系统管理员

前置条件：

- 更新包已经准备就绪

基本流：

1. 执行预更新检查：检查系统状态，内存与CPU资源是否充足
2. 进入维护模式：在游戏内进行全局操作，通知玩家即将进行更新
3. 停止游戏服务：按顺序停止游戏服务器，后台服务等
4. 部署更新内容：将更新的资源包部署到目标服务器上
5. 预览与自我检查：对更新完毕的游戏进行预览和自检

{系统运行}

6. 恢复游戏服务：按顺序恢复游戏服务器，后台服务等
7. 更新完成通知：通知所有玩家群体本次更新已结束
8. 更新内容文档化：记录本次更新的详细信息（时间，版本号等）
9. 后续等待小规模bug反馈
10. 结束

子流：

- S1执行灰度更新：将一小撮玩家部署到新版本验证更新内容的可行性并且在此阶段不发布更新公告，验证完更新可行性后将更新内容应用到所有玩家上

备选流：

A1：“执行回滚流程”

扩展点及触发条件：在基本流中任何情况下均可触发

事件流：

1. 停止有问题的更新版本服务，从备份中恢复被修改的配置文件或数据（或执行数据库回滚脚本）

2. 重新部署和启动上一个稳定版本的服务。

事件流恢复：对更新资源包进行检查修改，事件流恢复到基本流第4步

A2：“自检发现异常”

扩展点及触发条件：在{系统运行}扩展点发现游戏更新内容有错误，如bug，实际内容和预期不符

事件流：

1. 执行备选流A1

事件流恢复：同备选流A1恢复

“A3：紧急热修复”

扩展点及触发条件：在{后续等待小规模bug反馈}扩展点获取到玩家反馈

事件流：

- 1. 准备最小化的修复补丁
- 2. 执行基本流，跳过部分非必要的步骤

事件流恢复：事件流恢复到基本流第9步

反作弊监控

参与者：系统管理员

前置条件：

- 监控系统已经部署并运行

基本流：

- 1. 日常监控与巡检
 - 系统管理员查看核心指标的整体健康状况。
 - 关注异常峰值：如特定API错误率飙升、来自特定IP区域的登录激增、游戏对局数据异常等。
- 2. 告警触发：监控系统基于预设规则自动触发告警。
- 3. 初步分析与调查
 - 系统管理员查看告警详情，关联相关日志进行深入分析。
 - 操作包括：
 - 查询该玩家的详细行为日志（登录、对局、物品变动）。
 - 分析网络数据包捕获（如有）。
 - 检查该玩家客户端的反作弊心跳报告或扫描结果。
 - 关联其他数据，判断是误报还是确凿的作弊行为。

4. 分类与判定

- 根据分析结果，对事件进行分类：
 - 确认作弊：证据确凿。
 - 高度可疑：需要进一步观察或上报安全工程师进行深度分析。
 - 误报：由网络延迟、游戏BUG或其他合法原因引起。

5. 执行标准响应

- 对于【确认作弊】：
 - 根据预定义的策略，在游戏后台管理系统中执行封禁操作。封禁类型可能包括：账号封禁、硬件ID封禁、IP段封禁（谨慎使用）。
 - 记录封禁原因、证据和操作人员。
- 对于【高度可疑】：
 - 将玩家加入观察列表，进行更密集的日志记录。
 - 创建工单并转交给安全工程师（备选流 A-1）进行深入调查。

6. 记录与报告：在安全事件管理系统中完整记录本次事件、分析过程和处置结果。

7. 规则优化：根据本次事件中发现的误报（备选流 A-2）或新型作弊手段（备选流 A-3），提出优化检测规则的请求，并转交给安全或开发团队。

子流：

- S1进行玩家行为分析：需要对一个可疑玩家进行深度调查时，调取该玩家一段时间内的所有对局数据（K/D，爆头率，视角移动方式等），分析其历史数据是否有突然且不合理的表现提升，生成分析报告作为判断证据
- S2处理大规模作弊事件：检测到新型外挂在游戏内广泛传播时，立即在运营团队内部进行警告，安全团队分析新型外挂的特征，制定相应检测与封禁规则，严格封禁作弊玩家，公示封禁名单以维护游戏正义

备选流：

A1：“上报安全工程师”

扩展点及触发条件：在{执行标准响应}扩展点，发现有需要高度怀疑的玩家

事件流：

1. 反作弊团队收集相关信息给安全工程师
2. 安全工程师进行深度调查并给出报告

事件流恢复：回到基本流6，公布执行结果

A2：“处理误报”

扩展点及触发条件：在步骤4中确定为误报

事件流：

- 1. 解除对玩家的限制，如果导致玩家游戏体验受损，对玩家发出致歉邮件与补偿

事件流恢复：回到基本流7，汇报信息进行反作弊规则的优化

A3：“应对未知作弊手段”

扩展点及触发条件：反作弊监测系统没有检测到，但通过大量玩家举报和游戏社区反馈推测有新型作弊手段

事件流：

- 1. 通过玩家举报与社区反馈推测判断新型作弊手段
- 2. 制定临时反作弊规则部署到游戏
- 3. 进行作弊者封禁

事件流恢复：回到基本流7，汇报反作弊信息进行反作弊规则优化

A4：“反作弊系统本身宕机”

扩展点与触发条件：通过玩家群体与社区反馈发现反作弊系统宕机

事件流：

- 1. 团队内部将其视为最优先事项，分析系统宕机原因
- 2. 尝试重启运行反作弊系统
- 3. 如果有必要暂停游戏服务则进行服务器维护修理反作弊系统

事件流恢复：回到基本流6，发布玩家和社区发布反作弊维护公告

设置活动和任务奖励

参与者：运营人员

前置条件：

- 运营目标已明确：团队已经确定了本次活动的具体目标（如提升用户留存率）
- 资源已就位：美术和文案资源就位，活动与任务的奖励已在数据库准备好

基本流：

- 1. 登录管理后台：运营专员使用授权账号登录游戏运营管理后台。
- 2. 进入活动管理模块：在后台中使用“活动管理”或“任务管理”功能模块。
- 3. 配置基础信息
 - 输入活动名称、活动类型（如登录活动、充值活动、挑战任务等）。
 - 设置活动时间：包括开始时间与结束时间（精确到分钟）。

- 设置活动描述和规则说明。
 - 设置活动或任务的完成条件。
 - 设置对应的奖励信息（S1，S2）。
4. 预览与自我检查：对设置好的任务与活动进行前端展示，确认是否无BUG存在。
 5. 发布活动：系统将活动状态变为“已发布”。活动在预设的开始时间自动对玩家生效。
 6. 记录活动或任务的日志：记录本次活动任务的详细信息。

子流：

- S1配置多档位奖励：适用于充值返利、累计登录等多档位活动。
- S2配置概率型奖励：适用于开箱、抽奖类活动。

备选流：

A1：“自检时发现错误”

扩展点及触发条件：在步骤4发现有错误存在

事件流：

1. 运营人员自己对发现的错误进行修正

事件流恢复：回到基本流4继续进行自检

A2：“活动上线后发现错误”

扩展点及触发条件：在活动正式上线后，由玩家群体发现有错误存在

事件流：

1. 运营团队将活动和任务紧急关闭
2. 由技术人员对存在的错误进行处理
3. 完成热更新

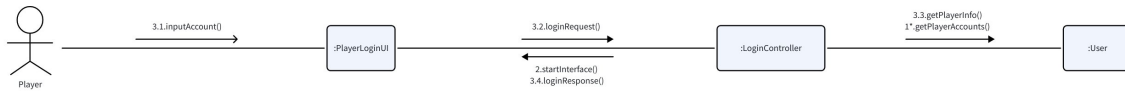
事件流恢复：回到基本流4进行自检

4. 需求分析

4.1 用况实现

4.1.1 游玩游戏

4.1.1.1 登录



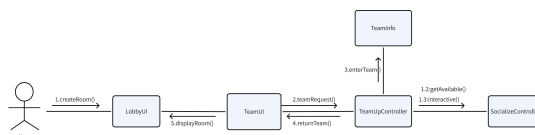
初步发现类： PlayerLoginUI、 LoginController、 User

- 实体类： User
- 边界类： PlayerLoginUI
- 控制类： LoginController

交互描述：

1. 控制器在系统初始化阶段主动向 Player 实体请求全部玩家账号数据，用于加载缓存
2. 控制器指示 UI 显示登录界面，玩家开始能够输入账号信息
3. 玩家在登录界面输入账号和密码
4. UI 将输入的账号与密码打包成请求，传递给控制器执行登录流程
5. 控制器根据玩家输入的账号信息进行匹配
6. 控制器将验证后的结果返回给 UI，UI 显示登录结果

4.1.1.2 组队



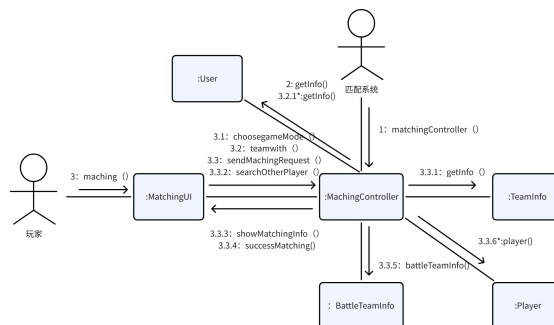
初步发现类： LobbyUI、 TeamUpController、 SocializeController

- 实体类： Player
- 边界类: LobbyUI、 TeamUI
- 控制类： TeamUpController、 SocializeController

交互描述：

1. 玩家在大厅界面点击组队按钮发出房间请求，组队控制器接收队伍请求
2. 组队控制器向社交系统请求空闲队伍或玩家数据
3. 玩家在队伍界面对社交控制器提供的数据进行交互
4. 组队控制器进行交互数据的处理，并将数据录入组队信息
5. 社交系统修改玩家或队伍状态
6. 组队控制器返回处理给组队界面，组队界面显示组队结果

4.1.1.3 匹配



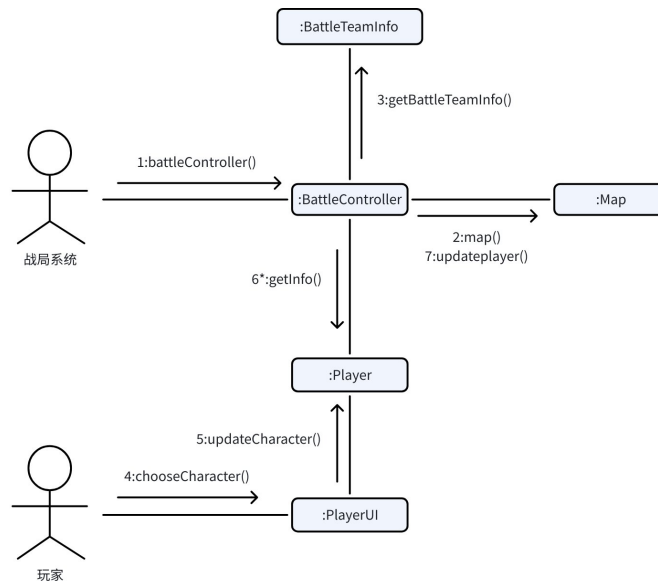
初步发现类： MatchingUI、 MatchingController、 TeamInfo,user,player

- 实体类： TeamInfo,user,player
- 实体类： BattleTeamInfo
- 边界类： MatchingUI
- 控制类： MatchingController

交互描述：

1. 玩家进入游戏，匹配控制器实例化/1/
2. 战局控制器从大厅User读取用户信息/2/
3. 玩家可以点击匹配/3/，控制对象获得玩家在匹配界面的操作信息（游戏模式选择）/3.1/（是否组队，队伍成员，成员信息）/3.2/，发送匹配请求/3.3/
4. 信息被封装在teamInfo当中/3.3.1/，以待发出，控制器寻找其他相同模式匹配玩家/3.3.2/
5. 匹配界面展示匹配情况/3.3.3/，是否匹配成功/3.3.4/
6. 成功后生成战局所有队伍的信息/3.3.5/
7. 实例化player/3.3.6/

4.1.1.4 生成对局



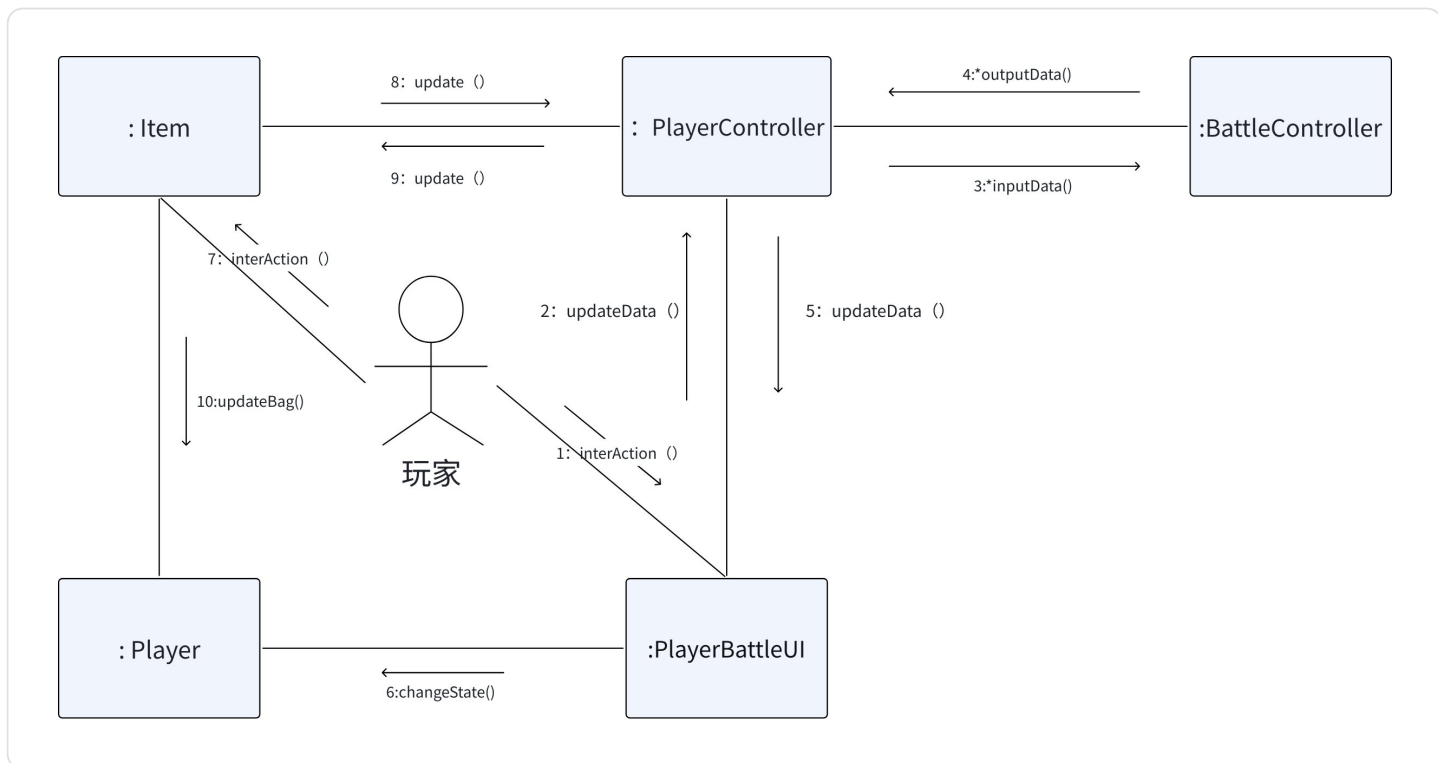
初步发现类： PlayerUI、 Player、 BattleController、 Map、 BattleTeamInfo

- 实体类： BattleTeamInfo、 Map、 Player
- 边界类： PlayerUI
- 控制类： BattleController

交互描述

1. 战局系统实例化战局控制器/1/，战局控制器实例化对局，战局加载地图/2/，然后从战局玩家列表中获取当前对局的所有玩家/3/
2. 玩家通过UI接口选择角色/4/，UI接口更新玩家的角色/5/，战局控制器获取玩家信息/6/
3. 然后战局控制器让对局更新对应的玩家/7/

4.1.1.5 战局内交互



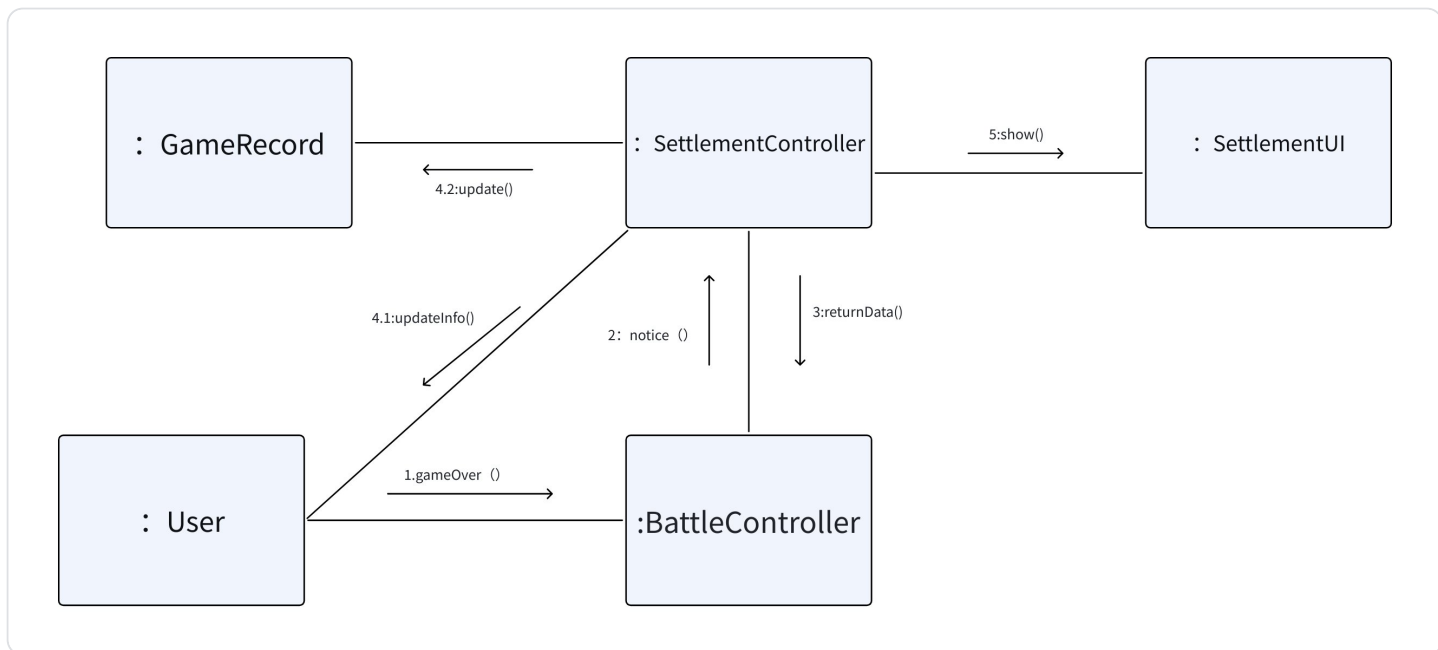
初步发现类：Player PlayerBattleUI PlayerController BattleController Item

- 实体类：Player Item
- 边界类：PlayerBattleUI
- 控制类：PlayerController BattleController

交互描述

1. 玩家进入游戏后进行移动，射击等操作/1/，UI将消息传送给角色控制器/2/，再由角色控制器与战局控制器进行数据交互/3/ /4/，返回结果给玩家UI/5/，变换玩家状态/6/
2. 玩家与游戏场景内的实体进行交互（如拾取道具，布置陷阱）/7/，实体将消息传送给角色控制器/8/，再由角色控制器与战局控制器进行数据交互/3/ /4/，返回结果/9/，更新玩家的背包/10/

4.1.1.6 结算对局



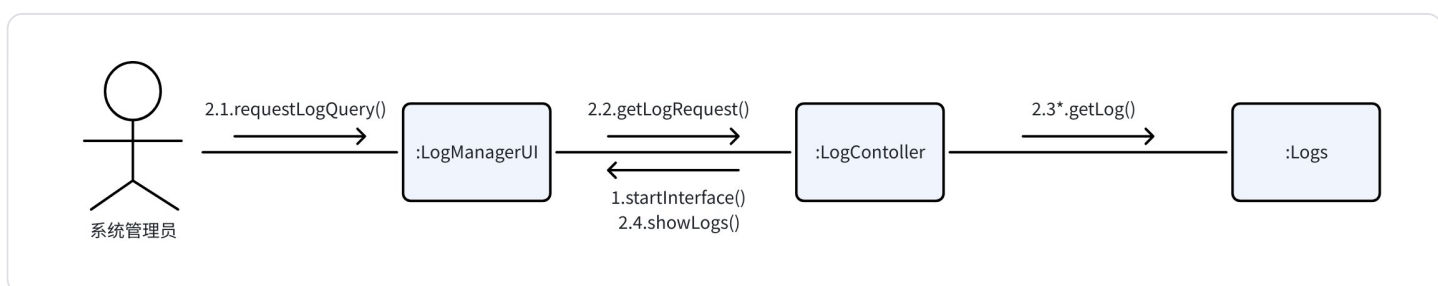
初步发现类： User SettlementController BattleController SettlementUI GameRecord

- 实体类：User GameRecord
- 边界类：SettlementUI
- 控制类：SettlementController BattleController

交互描述

1. 玩家结束对局，向战局控制器发送信息/1/，战局控制器通知结算控制器结算处理相关信息/2/并返回给战局控制器结果/3/
2. 结算控制器处理完对局信息后，更新玩家的游戏数据/4.1/，并更新游戏记录（战绩等）/4.2/
3. 结算控制器通过UI展示本局结算信息/5/

4.1.2 获取系统日志



初步发现类： LogManageUI、 LogController、 Logs

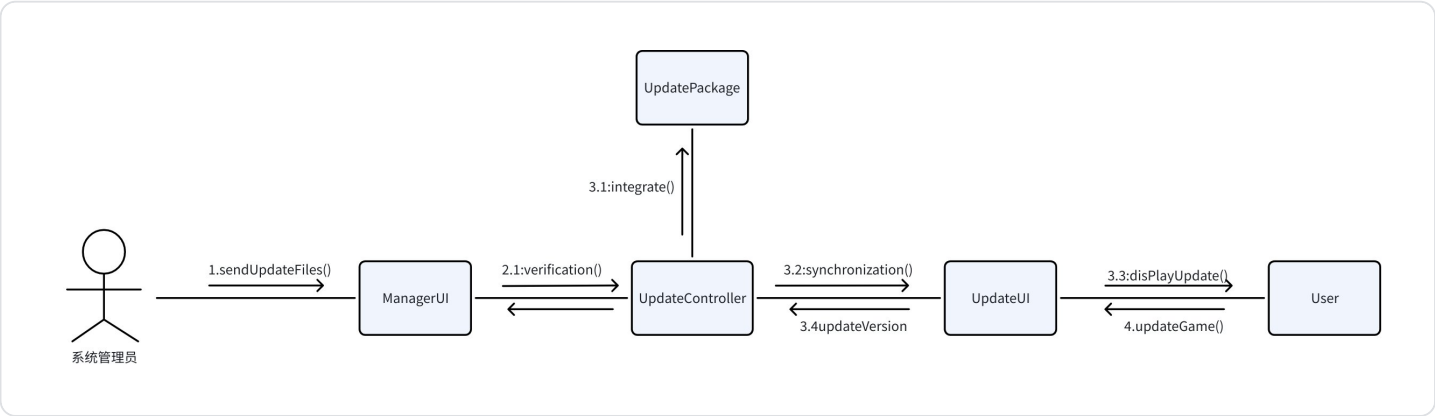
- 实体类：Logs
- 边界类：LogManagerUI
- 控制类：LogController

交互描述.

1. 控制器指示UI显示日志查询界面

- 2. 管理员在界面中输入筛选条件（时间范围、战局ID、类型等信息）并提交查询
- 3. UI将获取日志请求发送给日志控制器并附带相应的日志筛选条件
- 4. 控制器根据用户提交的筛选条件向系统日志实体请求匹配的日志记录
- 5. 日志实体返回查询到的日志数据列表
- 6. 控制器将请求结果返回给UI，UI进行显示对应的日志记录

4.1.3 更新内容



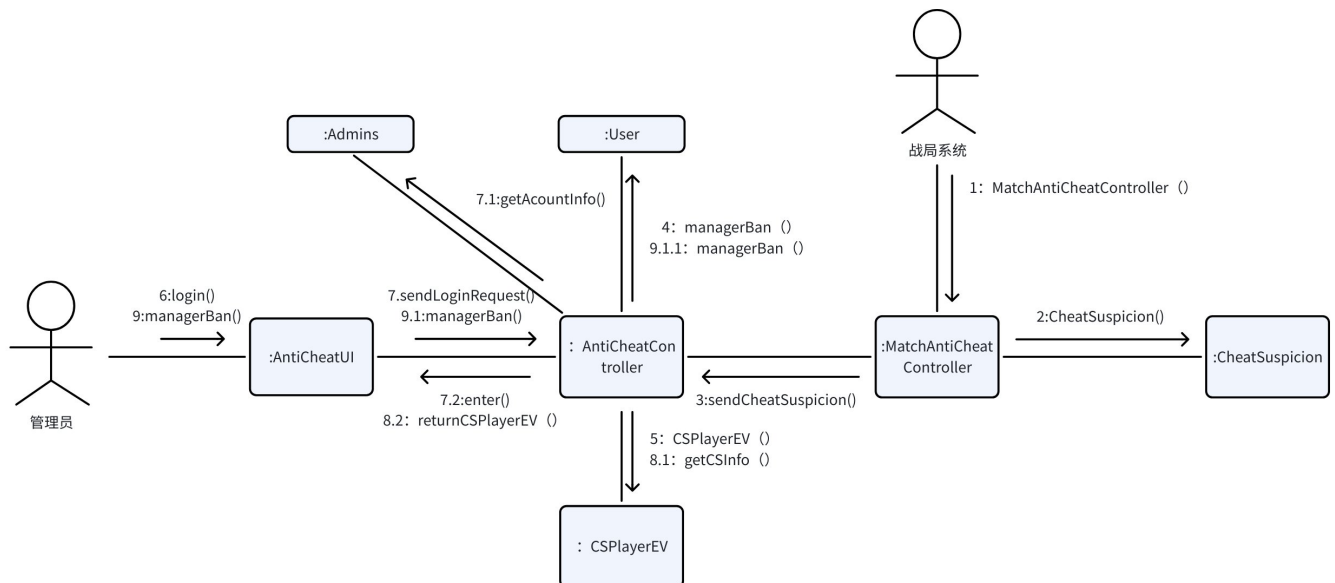
初步发现类：ManagerUI、UpdateUI、UpdateController、UpdatePackage

- 实体类：UpdatePackage、User
- 边界类：ManagerUI、UpdateUI
- 控制类：UpdateController

交互描述：

- 1. 系统管理员通过管理员UI发送更新文件
- 2. 系统管理员在UI界面向更新控制器发送验证申请，验证来源及版本是否一致，并让其返回版本号
- 3. 控制器将新旧文件合并打包并发布，更新UI同步更新内容
- 4. 游戏更新UI显示更新信息给玩家
- 5. 更新控制器更新版本

4.1.4 反作弊监控



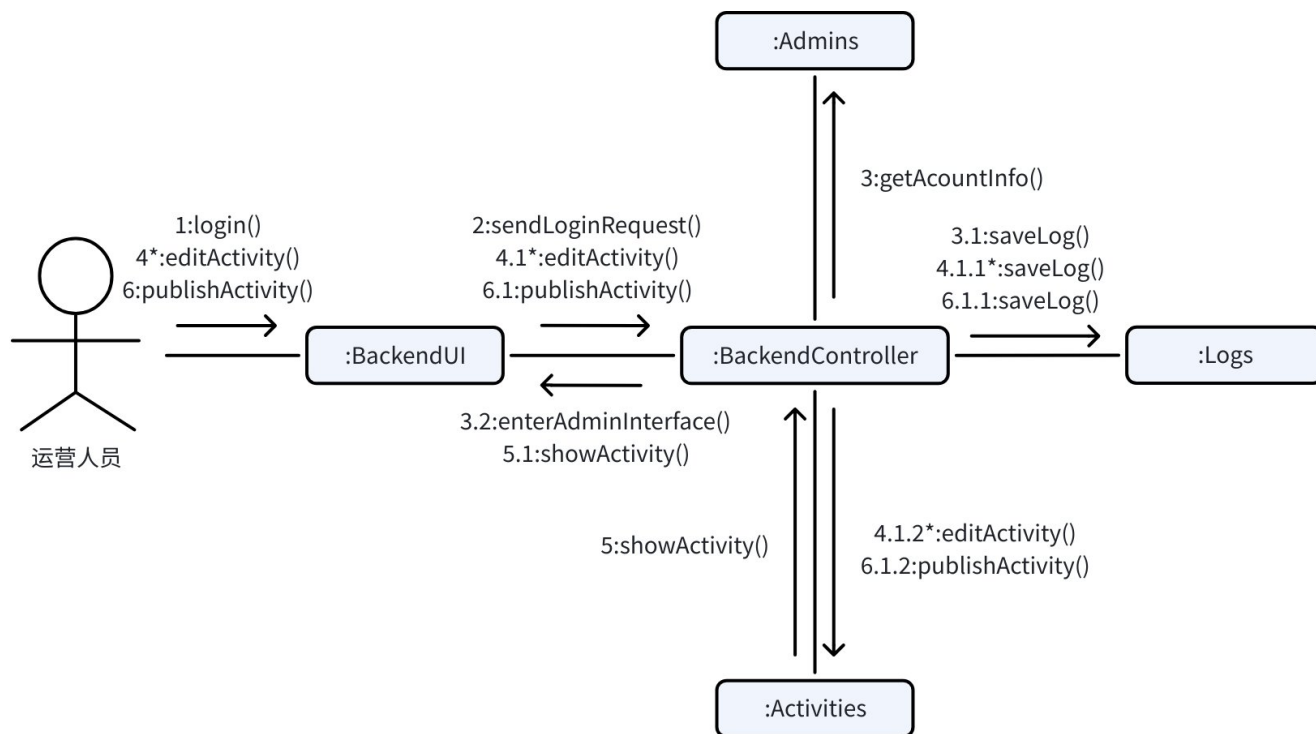
初步发现类：AntiCheatUI、AntiCheatController、CheatSuspicion，matchAntiCheatController、CSPlayerEV，admins

- 实体类：CheatSuspicion，CSPlayerEV，admins
- 边界类：AntiCheatUI
- 控制类：AntiCheatController，matchAntiCheatController

交互描述：

1. 当战局开始时，战局系统实例化局内反作弊控制器/1/
2. 局内反作弊控制器监测可疑活动，对应玩家生成“作弊嫌疑”数据/2/
3. 对局结束后，局内控制器传递可疑数据信息至系统反作弊控制器（包含可疑玩家信息，可疑操作数据）/3/
4. 系统反作弊控制器对典型确认作弊玩家封禁/4/，对疑似作弊行为封装为日志和玩家信息/5/
5. 管理员验证身份可登入管理员系统/6/。系统反作弊控制器接受登录请求/7/，获取账号信息并核对密码/7.1/，密码正确便允许进入反作弊管理界面/7.2/
6. 多次怀疑作弊的玩家和日志从CSPlayerEV读取/8.1/，发送到管理员界面人工审核/8.2/，管理员手动处理封禁玩家/9/。反作弊管理界面将处理封禁发送到系统反作弊控制器/9.1/，系统反作弊控制器进行对应的封禁处理/9.1.1/。

4.1.5 设置活动和任务奖励



初步发现类：BackendUI、BackendController、Activities、Logs、Admins

- 实体类：Logs、Admins、Activities
- 边界类：BackendUI
- 控制类：BackendController

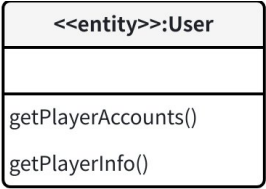
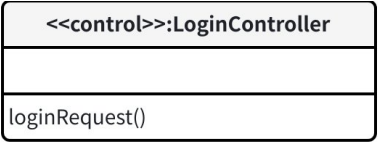
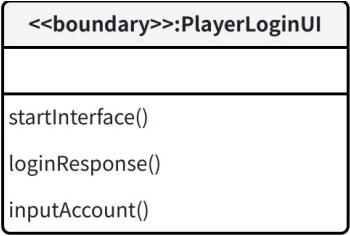
交互描述

1. 运营人员在后台界面登录/1/，后台界面向后台控制器发送登录请求/2/，后台控制器向Admins获取账号信息并对比密码/3/，然后将操作保存在日志中/3.1/，最后让后台界面进入管理界面/3.2/
2. 运营人员通过后台界面编辑活动/4/，后台界面将活动编辑信息发送给后台控制器/4.1/，后台控制器将操作保存在日志中/4.1.1/，然后编辑活动/4.1.2/，Activities将修改后的活动发送给后台控制器/5/，后台控制器将修改后的活动显示在后台界面以供核对/5.1/
3. 运营人员核对后发布活动/6/，后台界面将活动发布信息发送给后台控制器/6.1/，后台控制器将操作保存在日志中/6.1.1/，然后发布活动/6.1.2/

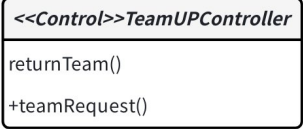
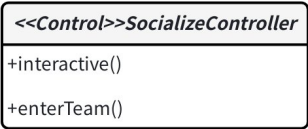
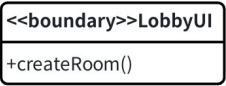
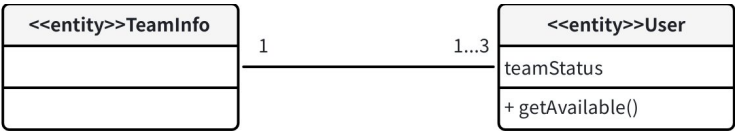
4.2 绘制类图

4.2.1 游玩游戏用况

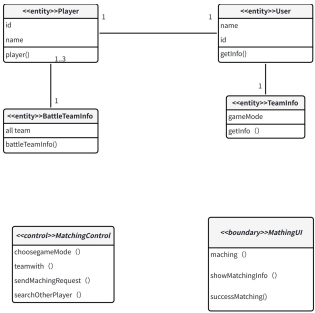
4.2.1.1 登录



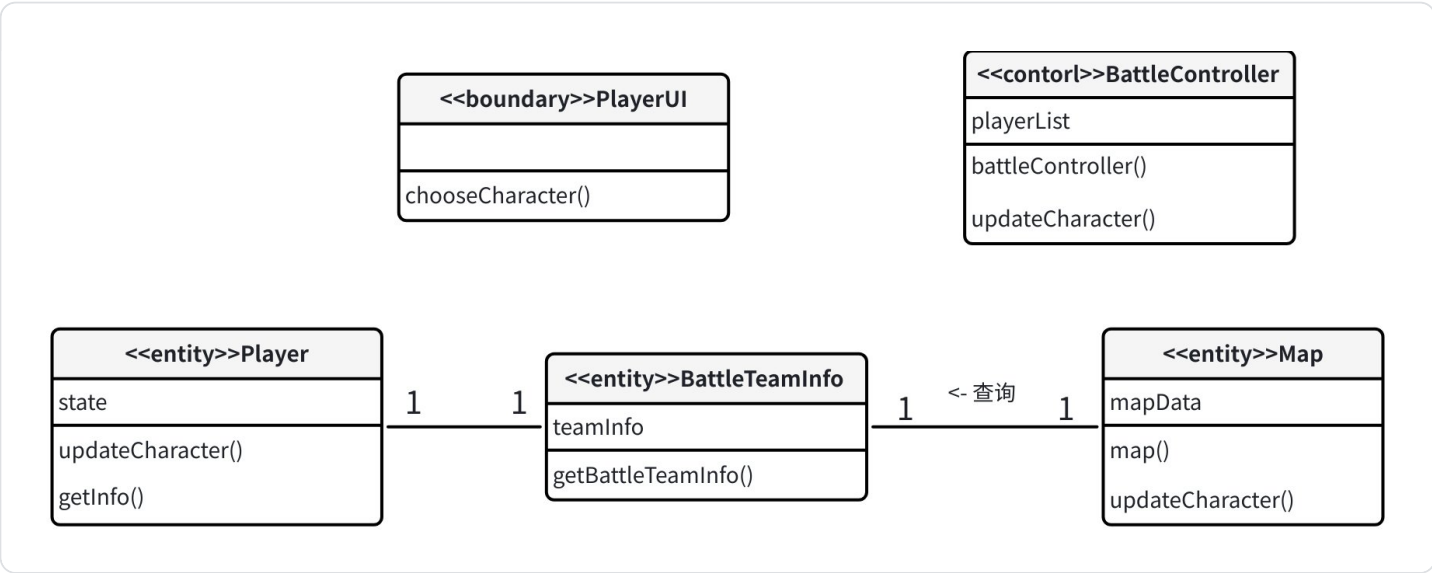
4.2.1.2 组队



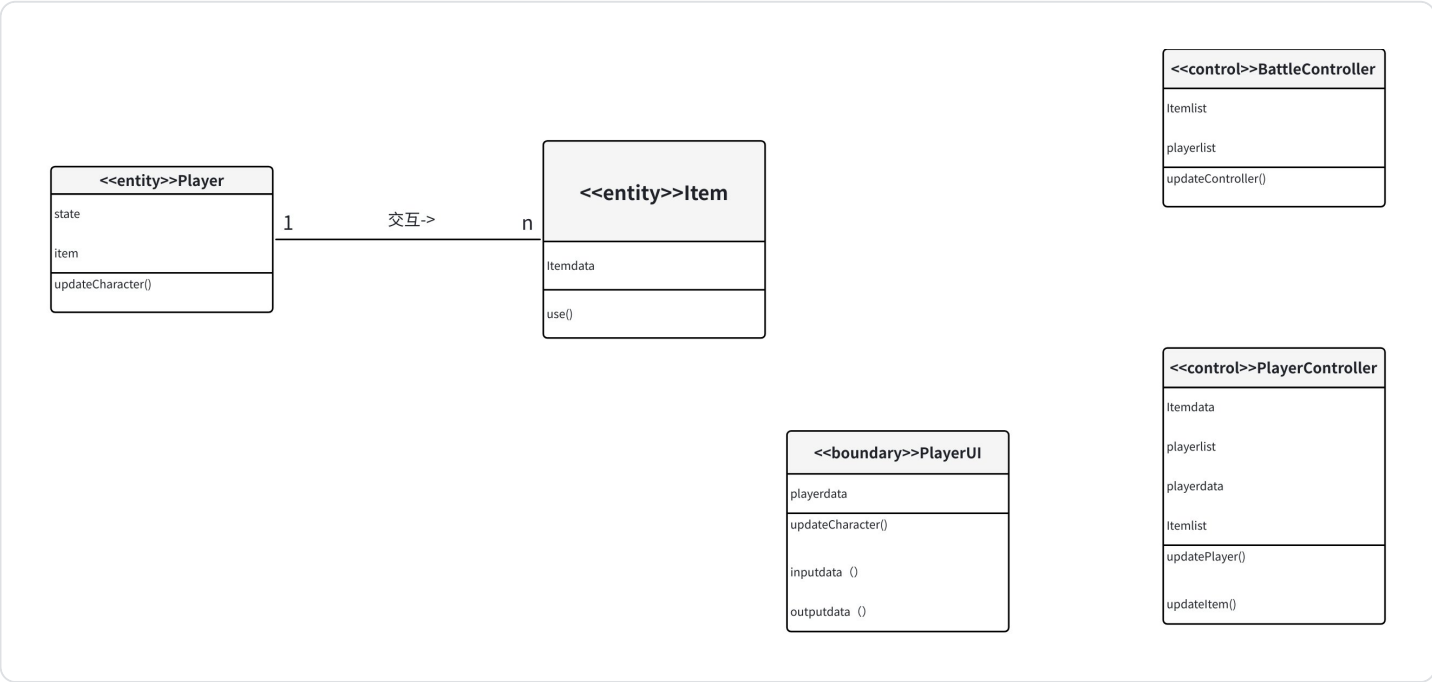
4.2.1.3 匹配



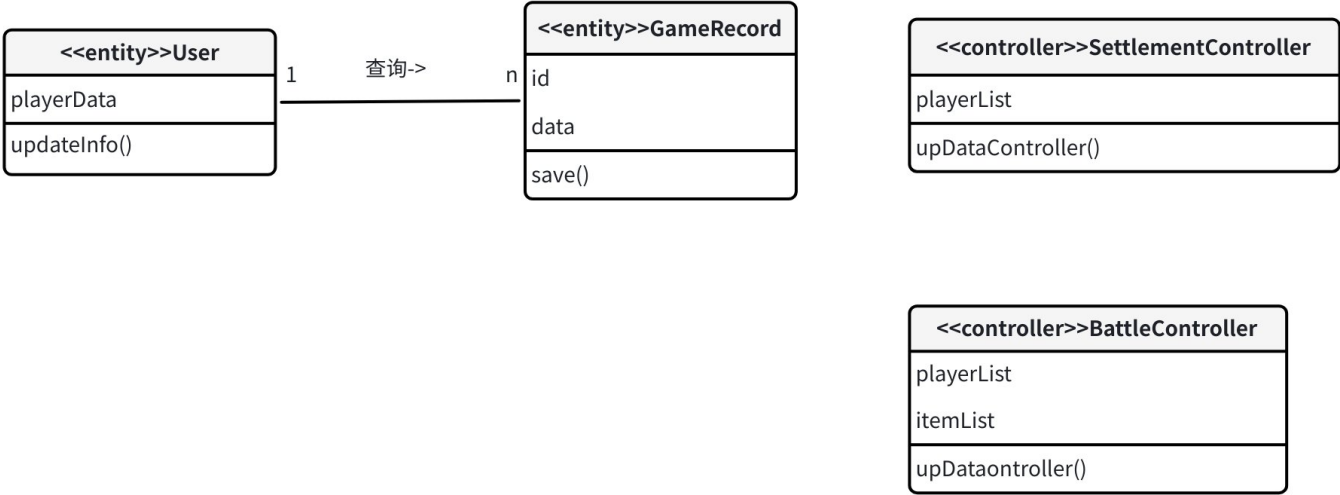
4.2.1.4 生成对局



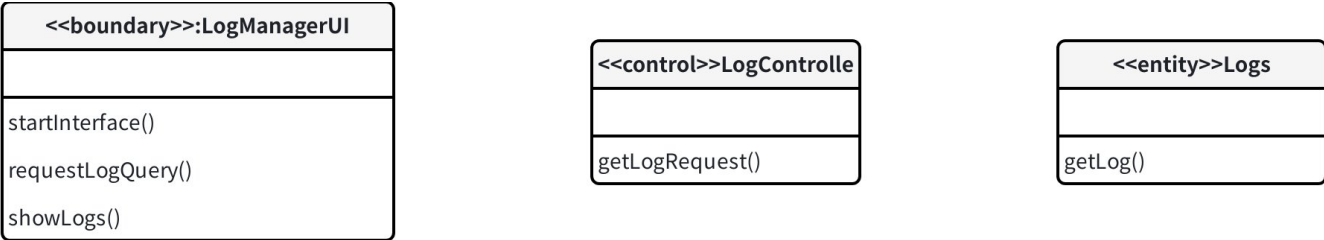
4.2.1.5 战局内交互



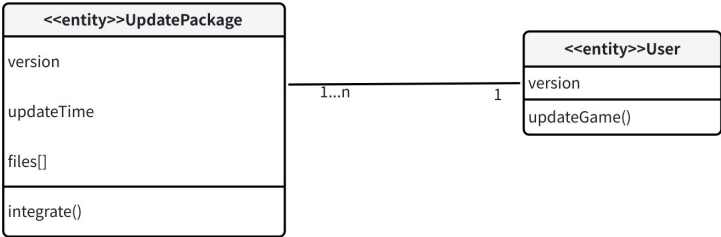
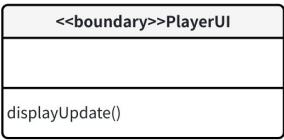
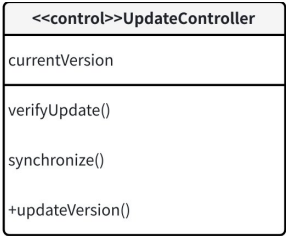
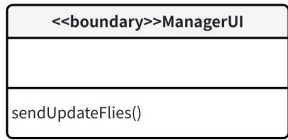
4.2.1.6 结算对局



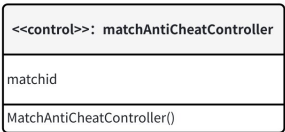
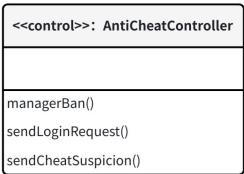
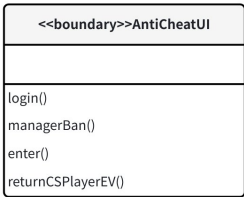
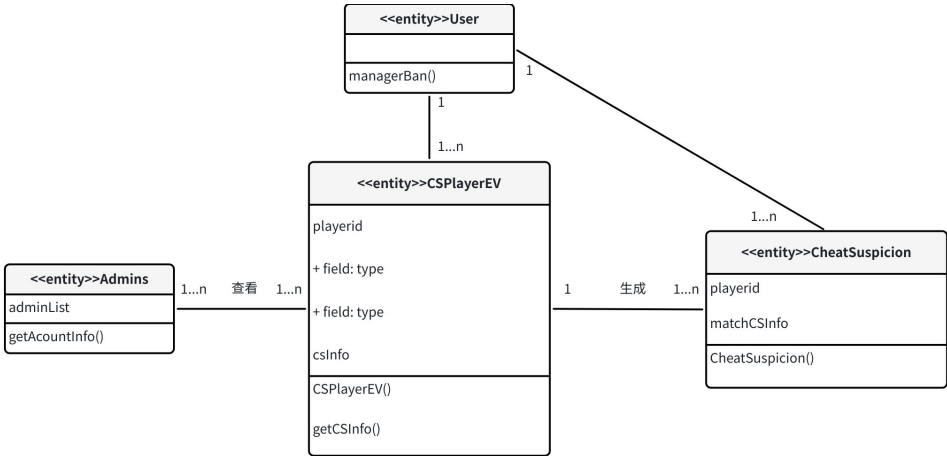
4.2.2 获取系统日志



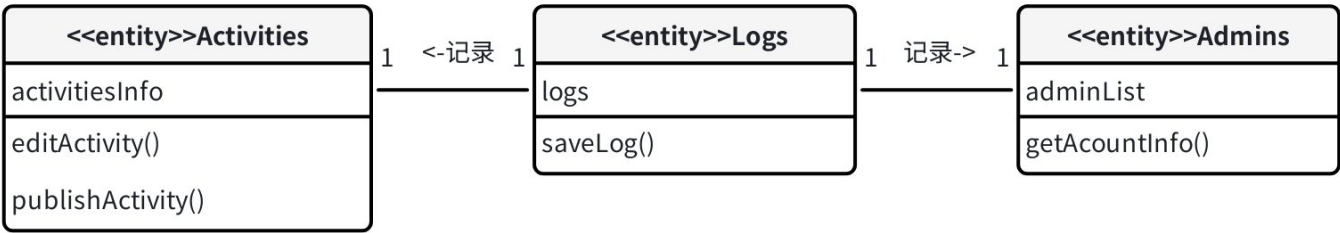
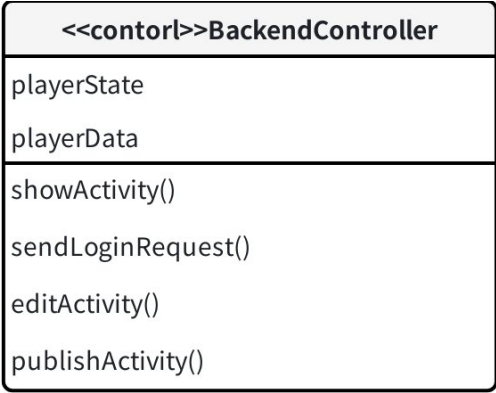
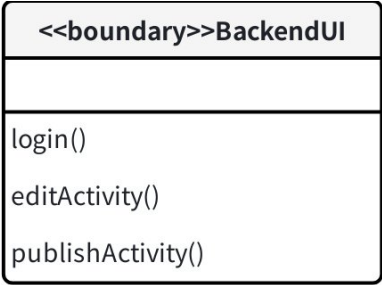
4.2.3 更新内容



4.2.4 反作弊监控



4.2.5 设置活动和任务奖励



4.3 职责分配与CRC卡

4.3.1 游玩游戏用况

4.3.1.1 登录

类名 PlayerLoginUI	
职责	协作
为玩家登录操作	

类名 LoginController	
职责	协作
协调登录逻辑，管理数据流	

4.3.1.2 组队

类名 LobbyUI	
职责	协作

为玩家提供大厅功能的接口及说明	提供TeamUI之间的切换
-----------------	---------------

类名 TeamUI	
职责	协作
为玩家提供组队功能的接口及说明	TeamUpcontroller提供的操作反馈至TeamUI

类名 TeamUpController	
职责	协作
为玩家执行组队相关操作	SocializeController提供交互信息及操作

类名 SocializeController	
职责	协作
实现玩家之间的社交交互	TeamUpController更改玩家状态并返回

4.3.1.3 匹配

类名 User	
职责	协作
提供玩家数据信息	

类名 Player	
职责	协作
实例化局内玩家	

类名 MatchingUI	
职责	协作
提供玩家（关于匹配）操作信息	提供玩家操作的信息（组队信息，游戏模式选择）

类名 BattleTeamInfo	
职责	协作
提供战局队伍信息	Matchingcontrol提供整合匹配成功所有队伍信息

类名 teamInfo	
职责	协作
提供队伍信息	MatchingUI提供玩家操作的信息（组队信息，游戏模式选择）

类名 Matchingcontrol	
职责	协作
整合匹配成功所有队伍信息	teamInfo提供队伍信息

4.3.1.4 生成对局

类名 PlayerUI	
职责	协作
提供当前玩家操作及状态	

类名 Player	
职责	协作
提供玩家状态数据	PlayerUI提供当前玩家操作

类名 BattleController	
职责	协作
更新玩家角色	Player提供玩家状态数据

类名 Map	

职责	协作
提供地图及角色数据	
提供玩家/敌人状态及位置	

类名 TeamInfoBattle	
职责	协作
提供队伍信息	

4.3.1.5 战局内交互

类名 PlayerUI	
职责	协作
监控玩家的操作与行为信息	PlayerController处理PlayerUI收集到的信息并反馈给UI

类名 PlayerController	
职责	协作
处理游戏内PlayerUI与Item的更新	BattleController监控PlayerController并实时跟进战局数据

类名 BattleController	
职责	协作
实时更新战局数据	PlayerController提供战局内玩家与物品的实时变化

类名 Player	
职责	协作
玩游戏	PlayerUI提供玩家操作的实时反馈，Item为玩家提供对局内的物品交互

类名 Item	
职责	协作
与玩家交互	PlayerController更新并监控场景上的物品数量与位置

4.3.1.6 结算对局

类名 User	
职责	协作
发出结算请求	BattleController接受玩家请求并开始处理

类名 BattleController	
职责	协作
接收玩家结算请求，向结算控制器获取对局信息	通知结算控制器并从中得到反馈

类名 SettlementController	
职责	协作
处理本次战局信息并输出对应数据	战局控制器接收对局信息，游戏记录保存战绩，结算UI展示结果

类名 SettlementUI	
职责	协作
展示本次对局结算信息	接受结算控制器的消息并被调用

类名 GameRecord	
职责	协作
保存玩家的对局信息（战绩等）	接受结算控制器的消息并被调用

4.3.2 获取系统日志

类名 LogManagerUI	
职责	协作
日志管理界面展示与交互	LogController返回查询结果

类名 LogController	
职责	协作
日志逻辑处理与流程协调	LogManagerUI展示查询界面

类名 Logs	
职责	协作
数据存储与基础查询	LogController传递筛选条件请求数据

4.3.3 更新内容

类名 UpdateController	
职责	协作
接收并验证更新文件	UpdatePackage检查文件完整性
同步更新到客户端	PlayerUI接收更新通知

类名 ManagerUI	
职责	协作
上传更新文件	UpdateController接收文件

类名 PlayerUI	
职责	协作
显示更新通知给玩家	

展示更新内容详情	UpdateController提供更新信息
----------	------------------------

类名 UpdatePackage	
职责	协作
提供更新包信息	
存储更新文件	
维护版本信息	UpdateController进行版本控制

4.3.4 反作弊监控

类名 User	
职责	协作
封禁玩家控制	Admins账号可以控制封禁玩家

类名 Admins	
职责	协作
提供管理员数据信息	

类名 AntiCheatUI	
职责	协作
列出需手动处理疑似作弊玩家信息	CSPlayer提供待手动处理疑似作弊玩家信息

类名 AntiCheatController	
职责	协作
整合CheatSuspicion数据，生成CSPlayer类	CheatSuspicion提供局内疑似作弊行为数据

类名 CSPlayer	
职责	协作

提供待手动处理疑似作弊玩家信息	AntiCheatController生成CSPlayer类
-----------------	--------------------------------

类名 matchAntiCheatController	
职责	协作
生成CheatSuspicion类	

类名 CheatSuspicion	
职责	协作
提供局内疑似作弊行为信息	matchAntiCheatController生成CheatSuspicion类

4.3.5 设置活动和任务奖励

类名 Admins	
职责	协作
提供所有管理员账号	

类名 Logs	
职责	协作
提供操作日志	

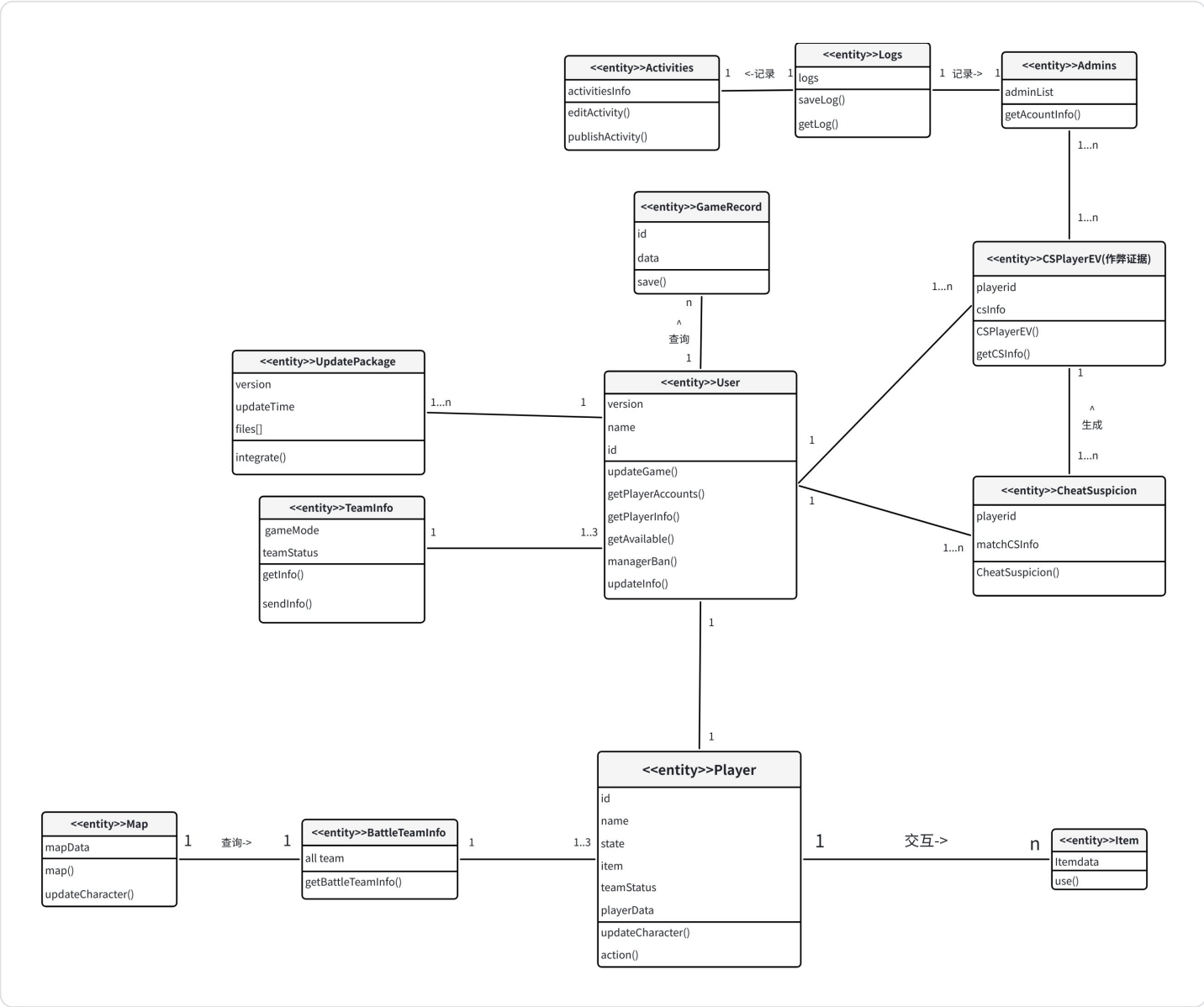
类名 Activities	
职责	协作
提供活动的信息	

类名 BackendUI	
职责	协作
提供登录	BackendController处理登录请求

编辑活动	BackendController编辑活动
发布活动	BackendController发布活动

类名 BackendController	
职责	协作
处理登录请求	
编辑活动	Activities提供活动的信息
发布活动	Activities提供活动的信息
显示活动	Activities提供活动的信息

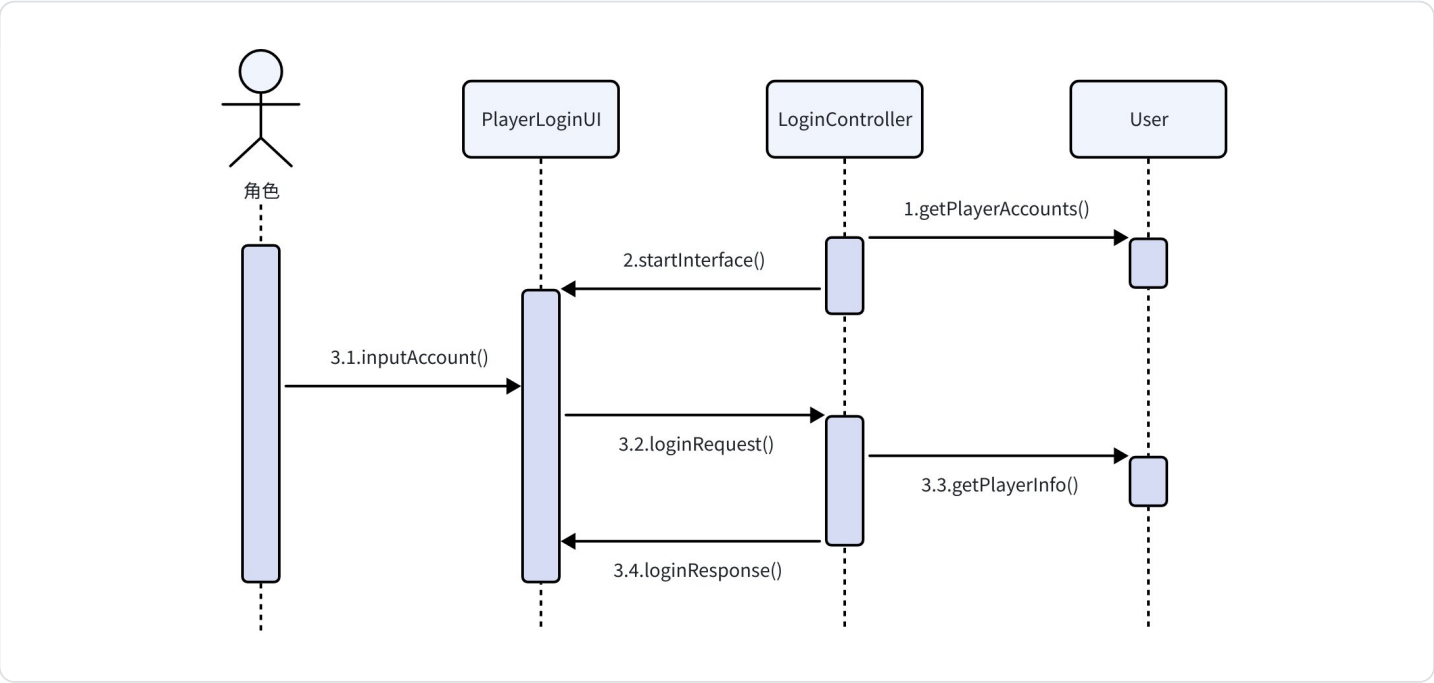
4.4 整合类图



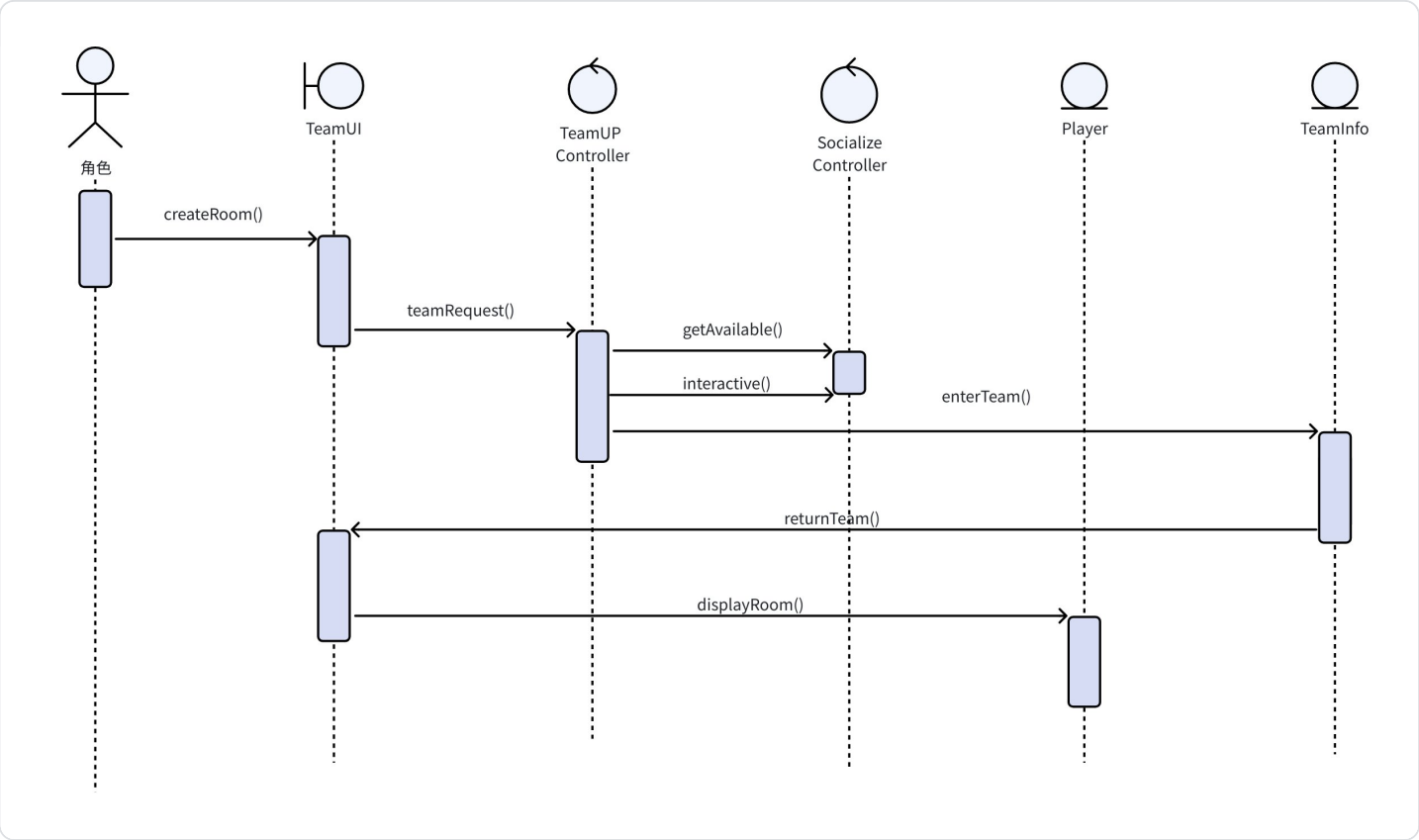
4.5 交互建模

4.5.1 游玩游戏用况

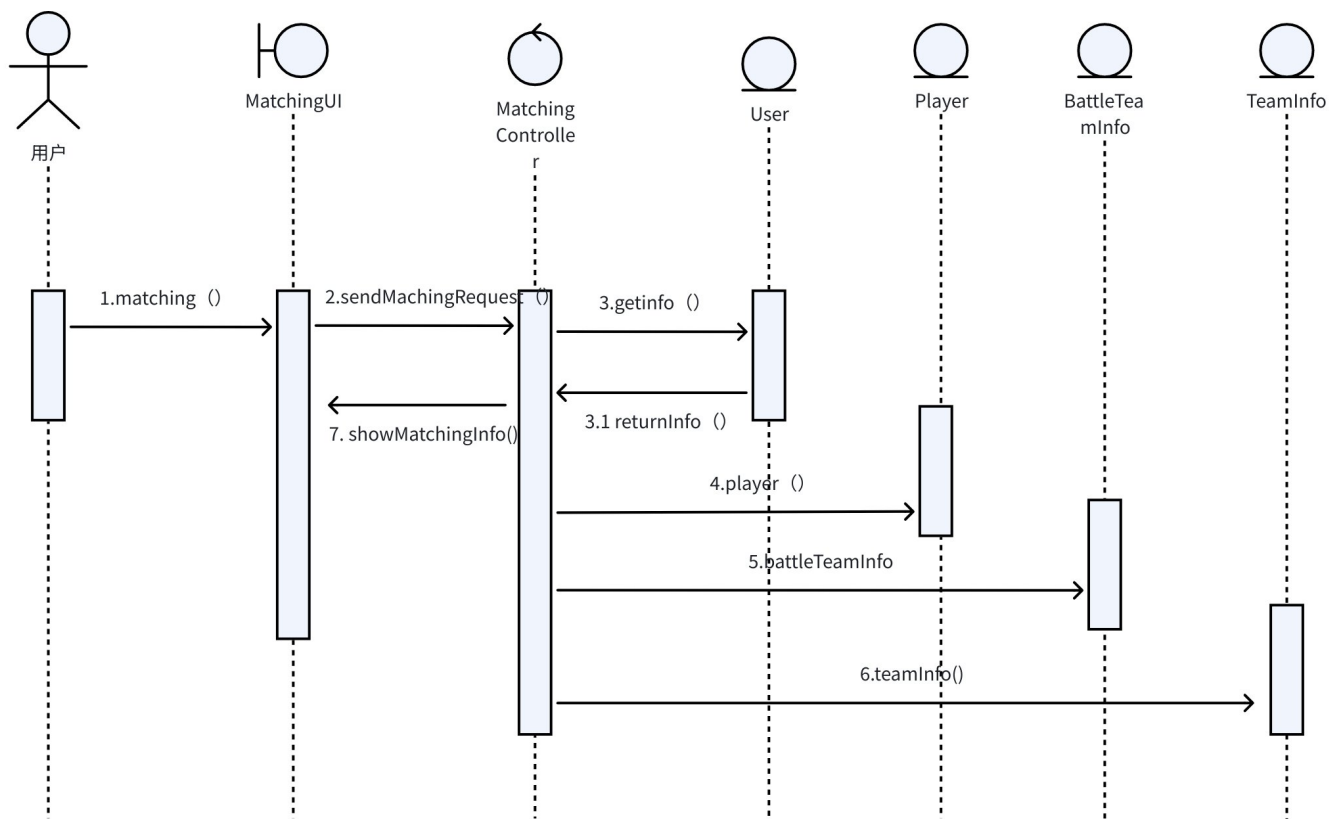
4.5.1.1 登录



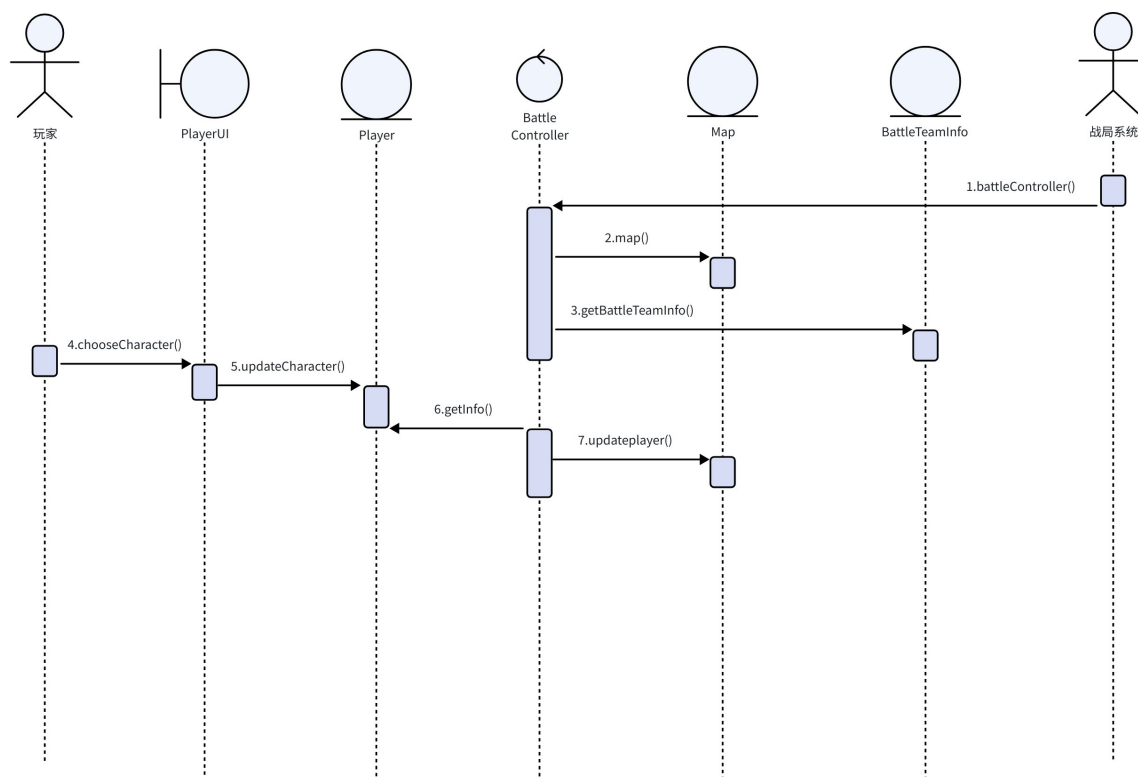
4.5.1.2 组队



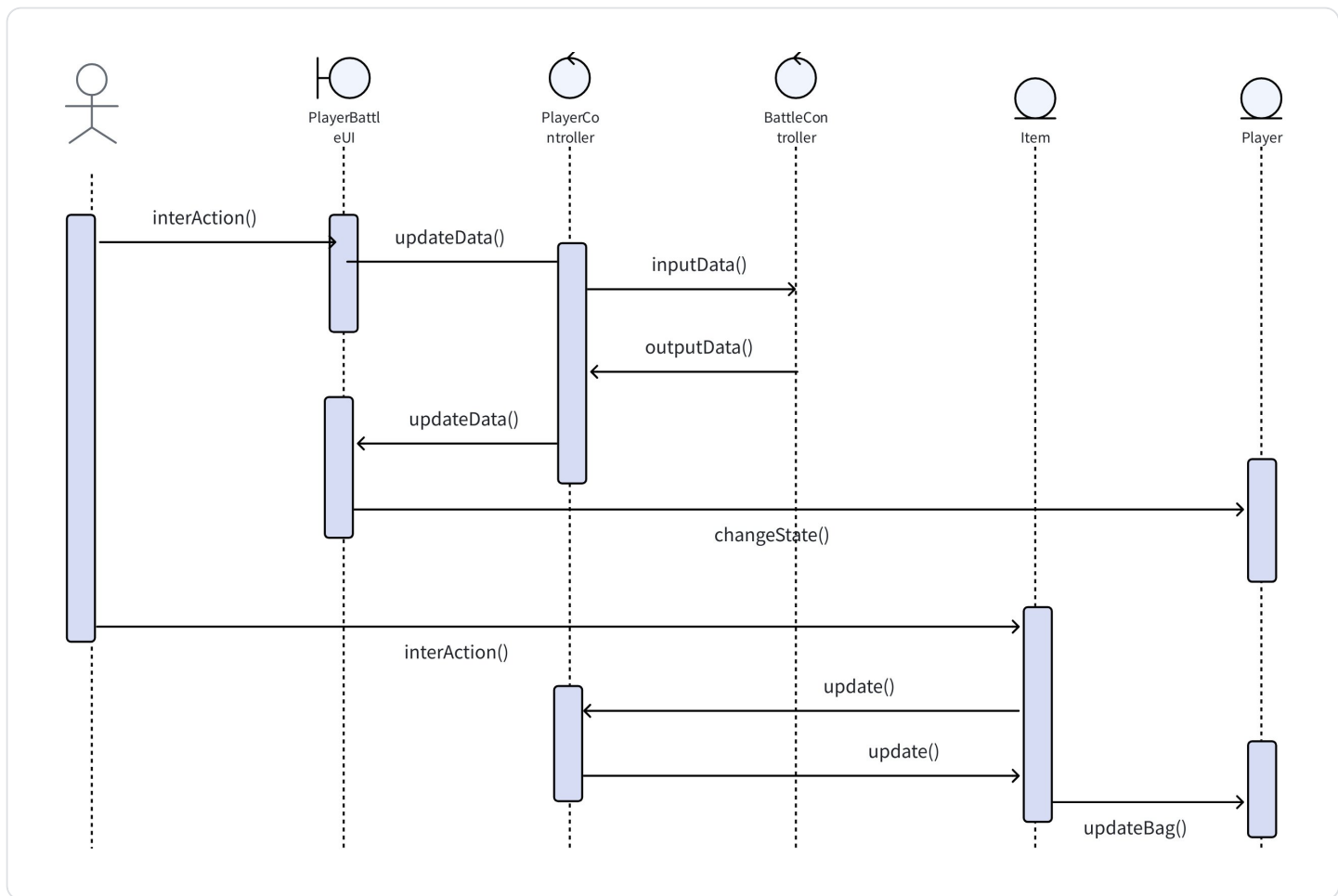
4.5.1.3 匹配



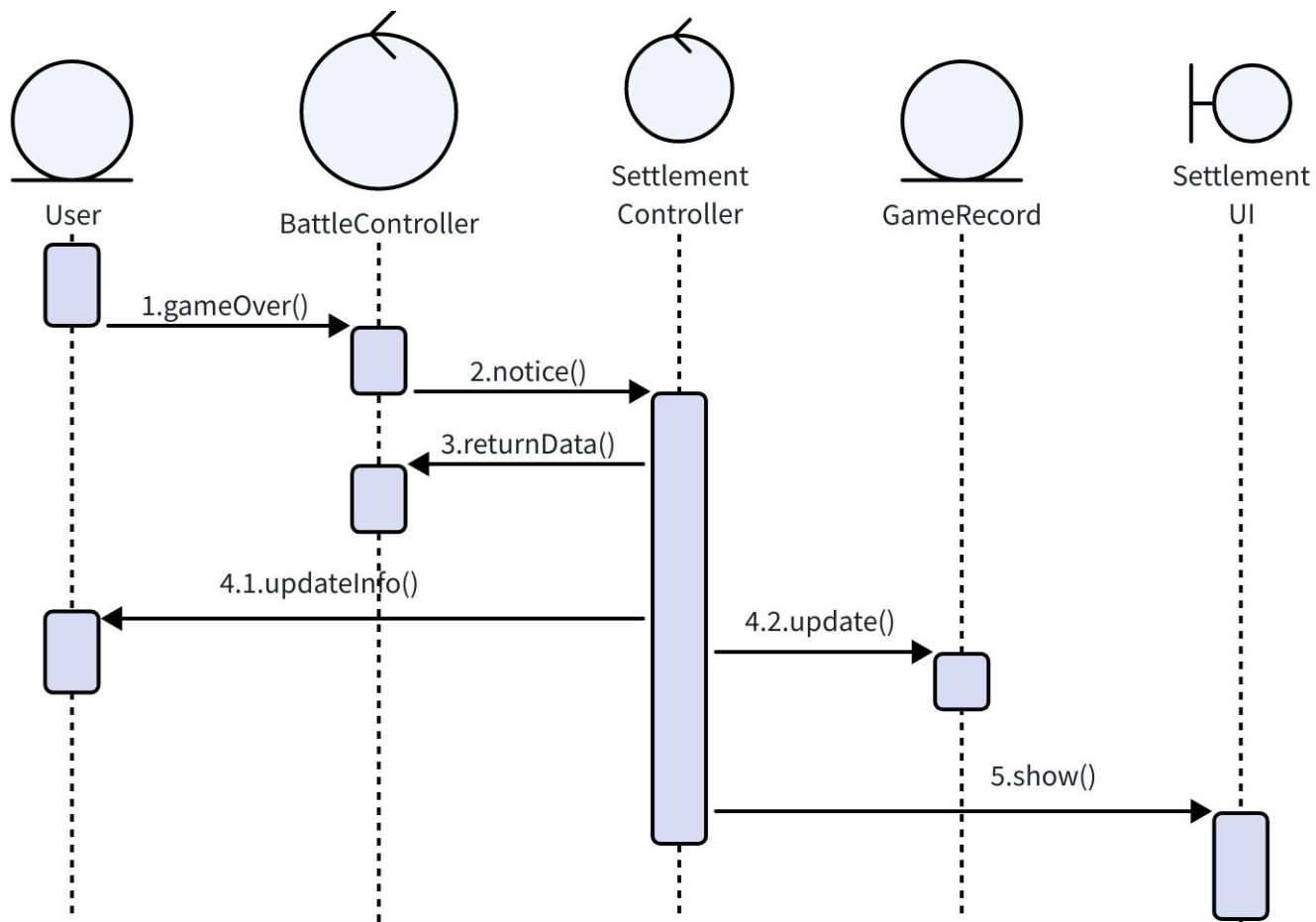
4.5.1.4 生成战局



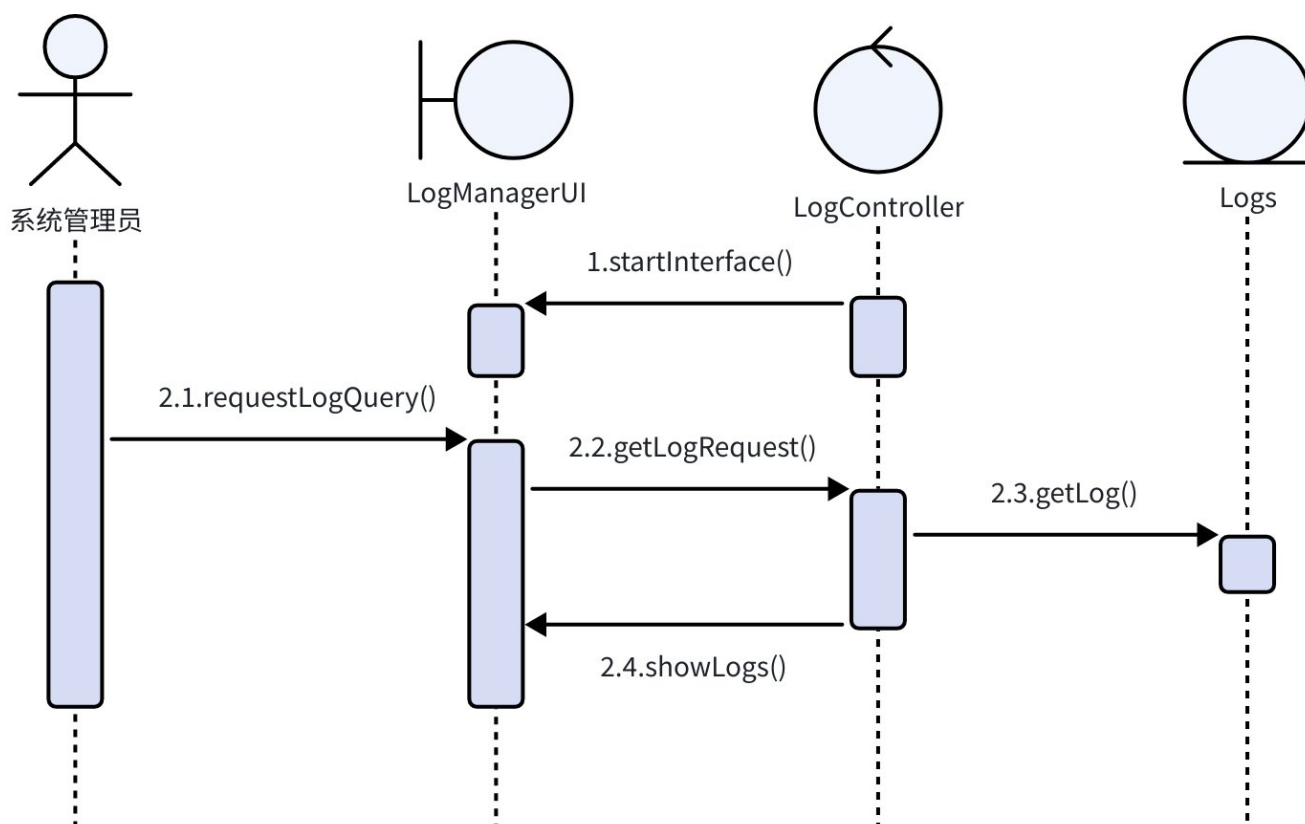
4.5.1.5 战局内交互



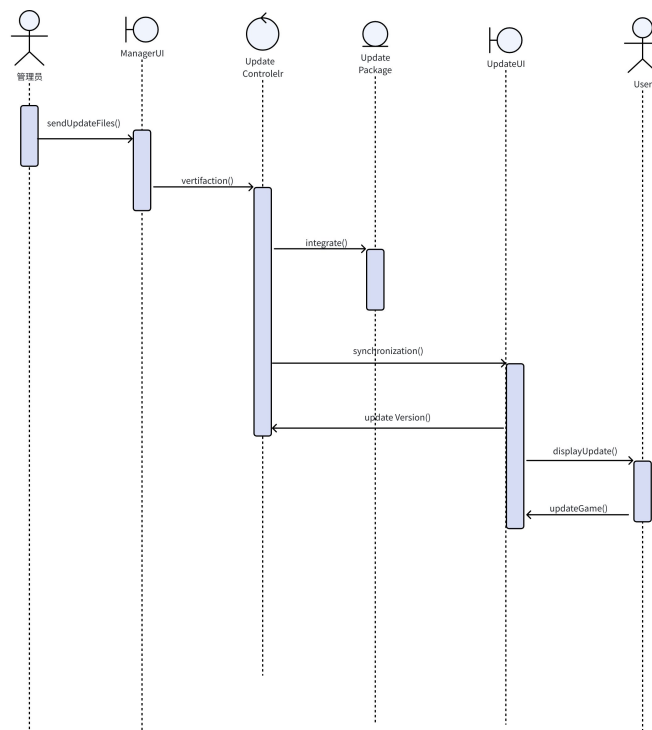
4.5.1.6 结算战局



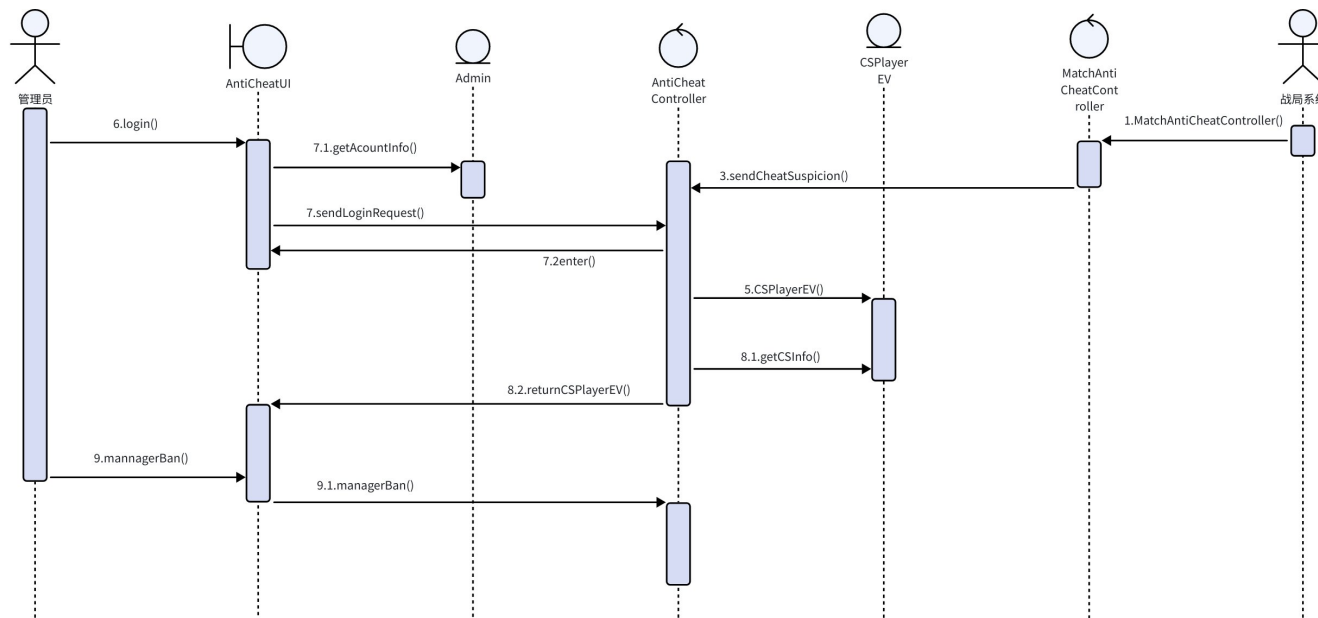
4.5.2 获取系统日志



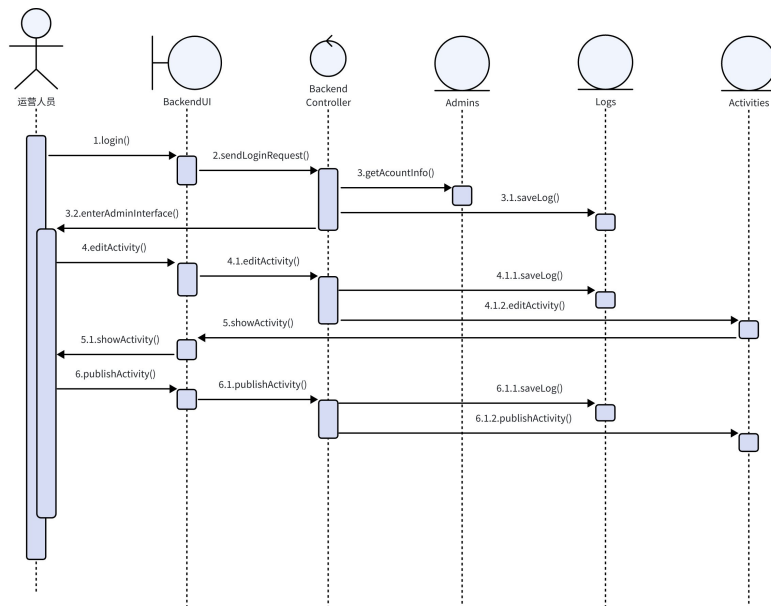
4.5.3 更新内容



4.5.4 反作弊监控



4.5.5 设置活动和任务奖励



5. 架构设计

5.1 设计原则

本系统在总体架构设计中遵循以下原则：

1. 分层解耦原则

将系统划分为网络通信层、业务逻辑层、数据管理层，明确各层职责边界，避免功能耦合，提高系统可维护性与扩展性。

2. 高并发与高实时性并重

针对多人实时在线联机游玩情况，采用异步消息分发与事件驱动机制，确保系统在高并发场景下仍具备稳定的响应能力。

3. 模块化与可扩展性

各模块相互独立，便于各功能模块的独立开发、测试与部署，便于团队协作与后续迭代

4. 事件驱动

采用事件降低各模块间的耦合度，提高系统响应性与可维护性

5. 安全与合规优先

确保游戏公平性、数据安全性与法律合规性

5.2 系统总体架构概述

系统整体采用**客户端—服务器（Client-Server）架构**，并在服务器端内部进一步划分为多个逻辑子模块。总体架构可分为以下四个核心层次：

1. 客户端表现层（Client Layer）
2. 网络通信层（Network Layer）
3. 业务逻辑处理层（Service / Logic Layer）
4. 数据与管理层（Data & Management Layer）

各层之间通过明确接口进行通信，降低模块间的直接依赖。

5.3 设计约束

- 技术栈约束：
 - 客户端：Unity+ C#
 - 服务器：C++
 - 通信协议：Protobuf+ TCP / UDP
- 平台约束：Windows平台优先
- 性能约束：
 - 客户端帧率：1080p分辨率下 $\geq 60\text{fps}$
 - 服务器CPU占用：单核心 $\leq 70\%$
 - 网络带宽：单玩家 $\leq 100\text{KB/s}$
- 开发约束：使用Git进行版本控制，遵循团队编码规范

5.4 客户端架构

5.4.1 模块划分

- 引擎 (Unity)
- 游戏逻辑
 - 角色及战斗系统
 - 角色控制器
 - 物品系统
 - UI系统
 - UIFramework
 - 界面管理器
- 网络通信

- Protobuf序列化/反序列化

5.4.2 技术栈

游戏引擎: Unity 6000.2.10f1

编程语言: C#

UI框架: Unity UGUI

网络库: 基于Socket的自定义封装

通信协议: Protobuf

音频: Unity Audio

动画: Unity Animation + Animator

5.5 服务端架构

5.5.1 网络通信层设计

5.5.1.1 网络通信模型

网络通信层负责客户端与服务器之间的数据传输，由于竞技游戏追求强实时反馈，因此采用多种通信方式：

- TCP通信：用于处理常规的注册、登录、游戏大厅组队等对可靠性要求较高的业务
- UDP通信：用于进行战局内游戏逻辑的实时同步业务

5.5.1.2 网络模块组成

网络通信层主要由以下模块构成：

模块名称	主要职责
TcpEndPoint	负责 TCP 连接的建立、维护与数据收发，处理客户端的连接
UdpEndPoint	负责实时消息的低延迟发送与接收，处理战局内状态同步
NetworkDispatcher	对网络消息进行序列化与反序列化处理，通过事件驱动分发网络消息

`NetworkDispatcher` 作为网络层与业务层之间的桥梁，根据消息类型将请求转发至对应业务模块，避免业务逻辑直接依赖底层网络实现

5.5.1.3 业务逻辑处理层

业务逻辑处理层是系统的核心，负责具体的功能与规则控制，主要分为以下子模块：

5.5.1.4 游戏服务器核心

`GameServer` 为系统核心控制类，负责统一管理服务器生命周期与各功能模块，处理各模块之间的信号连接事件，通过集中式管理，避免模块间直接强耦合

5.5.1.5 战局内逻辑处理模块

`GameRoomManager` 负责创建、处理、销毁各个战局实例，轮询获取各个战局生成的 `Tick` 事件，分发 `Tick` 事件交给网络模块处理，同步发送给各客户端

`GameRoom` 负责单个战局内的所有逻辑处理，维护独立的 `Tick` 节奏，是不同战局之间互不影响，提升并发性能

`PlayerEntity` 作为玩家在战局内的抽象实体，记录与处理玩家在战局内的状态

5.5.2 数据与管理层

`ClientInfo` 抽象玩家客户端为实体，将每一个客户端连接进行统一管理，实现登录层与逻辑层的解耦，便于处理断线重连等问题

`PlayerInfo` 存储玩家的实际数据，在客户端登录后，与客户端进行绑定

6. 详细设计

设计目标

项目结构

注：早期构建，与后续实际开发有些许出入

代码块

```
1  DeltaNights/
2  |—— Assets/                # Unity 主资源目录
3  |   |—— Animation/         # 动画资源
4  |   |   |—— AnimatorController/
5  |   |   |—— CharacterAnim/
6  |   |—— Meshes/            # 模型网格（模型需单独下载避免 Git 体积膨胀）
7  |   |—— Prefabs/           # 预制体
8  |   |   |—— UI/            # UI 单界面预制体
9  |   |—— Resources/         # 动态加载资源
10 |   |—— Scenes/            # 游戏场景
11 |   |—— ScriptableObjects/ # 数据配置
12 |   |   |—— UISettings/
13 |   |—— Scripts/           # 游戏逻辑脚本
```

```

14 | | | └─ FiniteStateMachine/
15 | | | └─ InputProcess/
16 | | | └─ PlayerControl/
17 | | |   | └─ PlayerFSM/
18 | | |   └─ PlayerStates/
19 | | └─ UIFramework/
20 | |   | └─ Core/
21 | |   | └─ Panel/
22 | |   | └─ UIAnimation/
23 | |   └─ Window/
24 | └─ Settings/           # 渲染管线等设置
25 | └─ Shader/            # 着色器
26 | └─ InputSystem_Actions.inputactions
27 |
28 | └─ DeltaNights-Server/  # 服务器代码
29 |   | └─ protobuf/       # Protobuf库
30 |   └─ CMakeList.txt/
31 |
32 | └─ Docs/               # 游戏开发文档
33 |   | └─ 涉众分析/
34 |   | └─ 特性分析/
35 |   | └─ 问题陈述/
36 |   └─ 愿景文档/
37 |
38 | └─ Packages/           # Unity Package 管理器内容
39 | └─ ProjectSettings/    # Unity 全局项目设置

```

功能描述：游戏战局房间管理类，处理房间的创建、销毁、玩家与房间之间的交互逻辑，获取每个房间的Tick事件，并传给网络分发层进行网络同步

局内玩家抽象

- playerentity.h

功能描述：战局内玩家抽象实体，记录客户端输入，计算玩家状态并同步给客户端

客户端管理

- clientmanager.h

功能描述：批量化管理连入客户端信息，登陆后将playerinfo（玩家账号数据）与clientinfo（连入客户端）做绑定，方便后续游戏内逻辑操作

- clientinfo.h

功能描述：将连入的客户信息进行实例化，提供信息使用接口，方便clientmanager进行管理

数据库

数据读取

- dataload.h

功能描述：数据读取类，数据读取核心模块

数据存储

- data.txt

功能描述：数据持久化核心，储存玩家账号基础数据，后续用PostgreSQL代替

- playerinfo.h

功能描述：服务器读取数据后实例化玩家账号数据供管理，是后续服务端与客户端交互的关键类

- playerinfomanager.h

功能描述：批量化管理playerinfo，提供接口与playerinfo交互，是实现服务端账号数据读取后对客户端响应进行逻辑操作的关键模块，如账号登陆等

客户端

战局处理系统

战局事件处理

- BattleManager

功能描述：处理战局事件

物品生成系统（待完成）

摄像机管理系统

- CameraManager

功能描述：用于统一管理游戏中玩家摄像机的状态切换，基于 Cinemachine 的优先级机制控制当前激活的虚拟摄像机

- GameCameraState

功能描述：用于标识游戏中摄像机的工作状态

事件分发系统

- Signals

功能描述：EventProcess 提供一套基于 Signal 的事件分发系统，用于实现模块间的低耦合通信

游戏设置

摄像机设置

- CameraSetting

功能描述：为摄像机系统的配置数据类，用于集中管理不同摄像机状态下的灵敏度与表现参数

全局数据设置

- GlobalSetting

功能描述：用于集中管理游戏中的全局配置数据，并在运行时为相关系统提供统一的参数访问入口

网络通信

ProtoTools

功能描述：protobuf的工具库，用于对象池封装，拓展方法，对象的快捷创建与重置

对象池

- MessageProcessor

功能描述：负责序列化与反序列化所有接收到的网络字节流，处理分发所有的网络事件，管理所有的网路同步事件

- NetWorkManager

功能描述：负责管理客户端的网络连接与消息通信，提供与服务器之间的基础数据收发能力。

TCP/UDP

- TcpManager

功能描述：负责处理所有的TCP网络传输事件，只负责TCP消息的发送与接受

- UdpManager

功能描述：负责处理所有的UDP网络传输事件，实现可靠的UDP以保证服务器逻辑同步的实时性与可靠性

控制系统

输入事件分发

- GameInput

功能描述：使用Cinemachine获取输入系统值，玩家输入捕获处理类

状态机

- PlayerFiniteStateMachine

功能描述：玩家状态机，初始化玩家状态

- PlayerState

功能描述：枚举存放玩家各类状态

角色控制系统

- PlayerController

功能描述：

负责玩家角色的基础移动与物理行为控制，包括位移、跳跃、重力、瞄准状态切换以及移动相关事件分发，是玩家行为的核心驱动模块。

- PlayerAimController

功能描述：负责玩家视角与角色朝向的联动控制，根据输入状态（普通 / 肩射 / 开镜）动态调整摄像机灵敏度与角色模型旋转行为。

- PlayerAnimationController

功能描述：负责玩家角色动画状态的驱动与平滑过渡，将 PlayerController 中的行为事件映射为 Animator 参数与动画层权重变化。

- PlayerAttackController

功能描述：负责将玩家输入抽象为攻击与技能事件，并协调武器系统与技能系统的执行逻辑，是战斗输入的核心调度模块。

- PlayerMeshController

功能描述：负责玩家模型的显示与隐藏控制，提供统一的模型管理接口，用于适配不同角色模型、第一人称 / 第三人称切换以及特殊视角表现需求。

- PlayerNetController

功能描述：

网络玩家控制器，用于映射网络玩家的所有操作到客户端角色上

- PlayerSkillController

功能描述：

玩家技能控制器，处理所有的技能逻辑

- PlayerWeaponController

功能描述：负责玩家武器的攻击逻辑控制，包括攻击触发、攻击冷却计算以及自动 / 半自动射击行为。

武器系统

- IWeapon

功能描述：定义武器系统的统一接口规范，约束所有武器必须具备的基础属性与攻击行为。

- WeaponData

功能描述：为武器系统的抽象数据基类，负责统一定义武器的基础配置与对外接口。

- WeaponType

功能描述：用于标识武器在武器系统中的分类类型。

- Bullet

功能描述：定义子弹的基本属性，初始化射击方向，进行碰撞检测。

技能系统

- ISkill

功能描述：定义技能系统的统一接口规范，约束所有技能必须具备的基础属性与生命周期行为。

- SkillConfig

功能描述：为技能系统的抽象配置类，用于以数据驱动方式定义技能的通用参数。

- SkillType

功能描述：用于标识技能在技能系统中的分类类型。

场景UI

UIFramework框架

- UIFramework

功能描述：声明所有的对外接口，充当UIManager的作用

- ASceneUIManager

功能描述：场景关卡UI管理基类，不同场景的UI管理器都继承自该类，该类用于管理所有UI事件回调

- Core

- 1. IUIController

功能描述：

UI界面控制器接口，声明所有UI界面控制器应有属性及方法，该类则为UI界面的逻辑层

- 2. IUIProperties

功能描述：UI界面属性接口，用于定义每个UI界面的属性，在开启UI界面时可以传入新的属性改变UI的显示逻辑

- 3. UIBaseLayer

功能描述：

基础UI界面的Layer，每个Layer层的子对象为需要管理的UI对象，通过访问管理对象的controller进行UI界面的控制

- 4. UIController

功能描述：UI界面控制器基类，用于实现单个UI界面的所有逻辑

- Panel

- 1. IPanelController

功能描述：面板UI控制器接口，在UI界面控制器接口的基础上声明面板UI特有属性与方法，实现该接口即为面板的逻辑层

- 2. IPanelProperties

功能描述：面板界面属性

- 3. PanelController

功能描述：面板控制器基类，控制面板的各种行为

- 4. PanelLayer

功能描述：面板（Panel）的Layer，通过访问管理对象的controller进行面板的控制，面板是一种没有历史记录、队列关系的UI界面，通过优先级划分多个子面板层，模块化管理不同优先级的面板层

- 5. PanelPriority

功能描述：定义面板的优先级

6. PanelPriorityLayerEntry

功能描述：面板优先级子层对象，定义各子层的属性

7. PanelPriorityLayerList

功能描述：根据优先级划分面板层的子层，使用列表存储各子层

8. PanelProperties

功能描述：面板界面属性基类，实现接口的具体定义

■ UIAnimation

1. AnimationView

功能描述：单个UI界面具体动画实现

2. AnimComponent

功能描述：UI界面动画组件，用于设置UI界面的过渡行为，继承该类并实现Animate方法用于实现UI界面动画的自定义，该类为动画处理器，需要单独挂载到场景上

3. FadeAnimation

功能描述：渐入动画

■ Window

1. IWindowController

功能描述：窗口UI控制器接口，在UI界面控制器的基础上声明窗口UI特有的属性与方法，实现该接口即为窗口的逻辑层

2. IWindowProperties

功能描述：窗口界面属性

3. WindowController

功能描述：窗口控制器，控制窗口的各种行为

4. WindowHistoryEntry

功能描述：窗口历史记录，用于存储历史窗口的信息，便于控制对应窗口

5. WindowLayer

功能描述：窗口（Window）的Layer，通过访问管理对象的controller进行窗口的控制，窗口是一种有历史记录和顺序的UI界面，通过栈和队列的结构对不同优先级的窗口进行存储

6. WindowPriority

功能描述：定义窗口的优先级

7. WindowPriorityLayer

功能描述：带蒙黑的辅助层，用于显示弹窗等窗口（如二次确认提示框等）

8. WindowProperties

功能描述：窗口属性基类，管理窗口各接口，暴露属性

登录UI

- StartUIManager

功能描述：游戏开始界面的UI管理器

- StartMainPanel

功能描述：游戏开始界面的Panel，目前无注册页

- LoginPanel

功能描述：游戏登录界面的Panel

大厅UI

- HallPanel

功能描述：游戏开始界面的大厅界面，仅显示在准备状态的房间

- PlayerListManager

功能描述：玩家列表管理器

- RoomListManager

功能描述：房间列表管理器

- RoomPanel

功能描述：房间的Panel

- CreateRoomWindow

功能描述：创建房间窗口

战局UI

- GameSceneUIController

功能描述：对局内UI管理器，负责接收玩家事件并显示玩家数据与属性变化

- PlayerStatusPanel

功能描述：玩家状态面板，包括生命值，弹药量等

后记

关键技术及其重难点：

- 服务端和客户端状态同步及处理
- 运行中角色的状态机管理和各种输入处理
- 事件驱动，降低耦合度
- 基于Qt的多线程事件驱动服务器架构
- TCP + UDP 双通道通信模型
- 网络线程与逻辑线程解耦
- 房间（GameRoom）驱动的Tick 逻辑模型
- Protobuf协议序列化

未来开发：

- 完成多人模式开发
- 实现搜打撤模式
- 完成异常流处理
- 完善游玩游戏用况和其他用况的实现
- 完善文档
- 服务端确定性物理检测
- 可靠UDP实现
- 状态同步的预测与回滚

参考文献

UML 2.2面向对象分析与设计(第4版)

UML 面向对象建模与设计